

Experiment No: 1

AIM : To understand various component of IoT tool Kit (such as Jumper wire, bread board, various sensors, and actuators).

OBJECTIVE : Gain familiarity with the components used in an IoT toolkit and understand their functions and applications in building IoT projects.

1. Jumper Wire

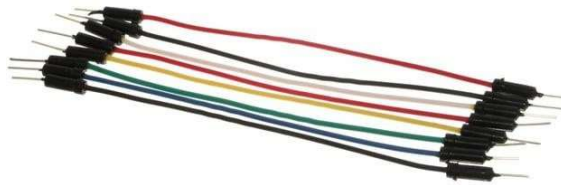


Figure 1.1

Description : A jumper wire is an electrical wire used to connect different components in a circuit without soldering. It is commonly used with breadboards, microcontrollers, and sensors to create temporary connections for prototyping. The wires are insulated and come in different colors for easy identification.

There are three main types of jumper wires: male-to-male, male-to-female, and female-to-female. Male connectors have pins, while female connectors have sockets. These wires come in different lengths and are essential for quick and flexible circuit assembly.

2. Breadboard

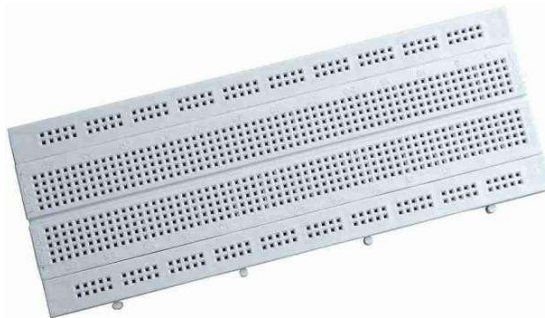


Figure 1.2

Description : A breadboard is a rectangular plastic board with many tiny holes used to build and test electronic circuits without soldering. It allows components like resistors, LEDs, sensors, and microcontrollers to be connected using jumper wires for easy prototyping. The holes are arranged in rows and columns, with metal strips underneath that connect them electrically.

Breadboards have two main sections: the power rails on the sides for providing power and the main grid in the center for connecting components. Since the connections are temporary, you can modify circuits easily, making it ideal for beginners and IoT projects.

3. Sensors



Figure 1.3

Description : A sensor is a device that detects changes in the environment and converts them into electrical signals for processing. Sensors are widely used in IoT projects to collect data such as temperature, motion, light, and humidity. They help smart devices interact with their surroundings by sending real-time information to microcontrollers like Arduino, ESP32, or Raspberry Pi.

There are different types of sensors, including temperature sensors (DHT11, DS18B20), motion sensors (PIR, Ultrasonic), gas sensors (MQ2, MQ135), and light sensors (LDR, TSL2561). These sensors play a key role in applications like home automation, weather monitoring, and industrial automation.

4. Actuators

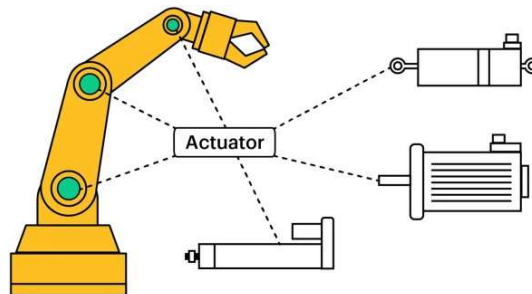


Figure 1.4

Description : An actuator is a device that receives signals from a microcontroller and performs a physical action, such as moving, turning on/off, or generating sound. Actuators are used in smart home devices, robotics, and industrial automation to control various functions based on sensor inputs.

Common types of actuators include motors (DC motor, stepper motor, servo motor) for movement, relays for switching high-power devices, buzzers for sound alerts, and solenoids for mechanical actions. They play a crucial role in IoT by enabling devices to interact with the physical world.

Experiment No: 2

AIM : To understand the Arduino Board architecture and bread board.

OBJECTIVE : Learn the structure, features, and pin configuration of Arduino UNO, and how to connect components using a breadboard.

1. Architecture of arduino uno board

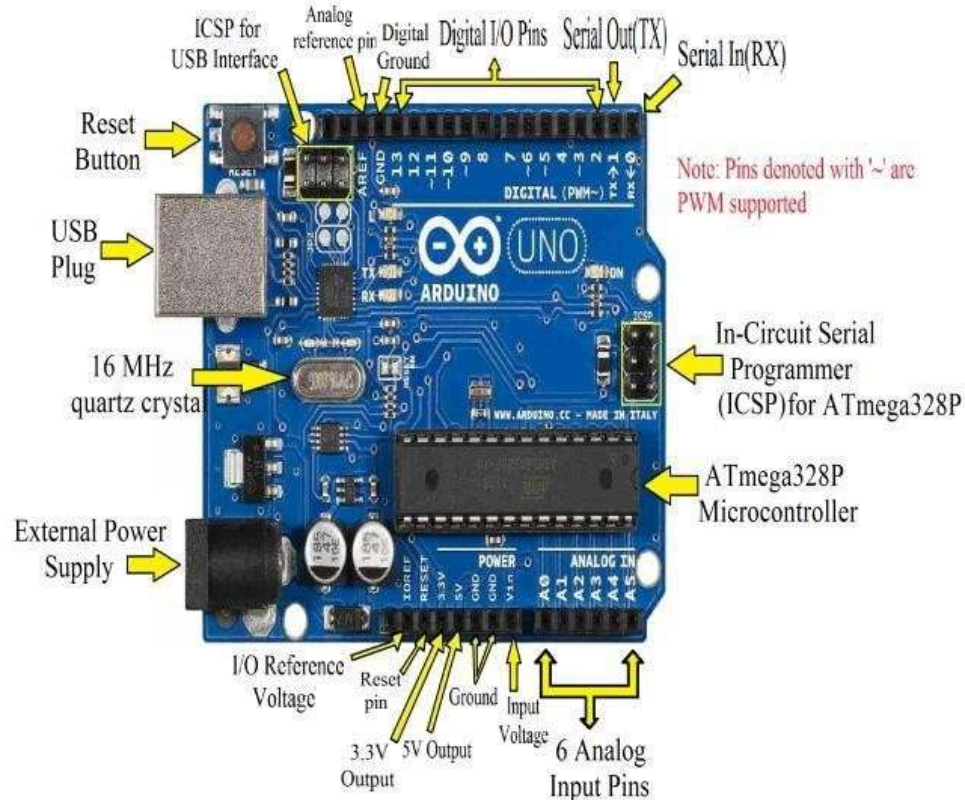


Figure 2.1

- **ATmega328 Microcontroller** – It is a single-chip Microcontroller of the ATmel family. The processor code inside it is of 8-bit. It combines Memory (SRAM, EEPROM, and Flash), Analog to Digital Converter, SPI serial ports, I/O lines, registers, timer, external and internal interrupts, and oscillator.
- **ICSP pin** – The In-Circuit Serial Programming pin allows the user to program using the firmware of the Arduino board.
- **Power LED Indicator** – The ON status of LED shows the power is activated. When the power is OFF, the LED will not light up.
- **Digital I/O pins** – The digital pins have the value HIGH or LOW. The pins numbered from D0 to D13 are digital pins.

- **TX and RX LED's** – The successful flow of data is represented by the lighting of these LED's.
- **AREF** – The Analog Reference (AREF) pin is used to feed a reference voltage to the Arduino UNO board from the external power supply.
- **Reset button** – It is used to add a Reset button to the connection.
- **USB** – It allows the board to connect to the computer. It is essential for the programming of the Arduino UNO board.
- **Crystal Oscillator** – The Crystal oscillator has a frequency of 16MHz, which makes the Arduino UNO a powerful board.
- **Voltage Regulator** – The voltage converter converts the input voltage to 5V.
- **GND- Ground pins** – The ground pin acts as a pin with zero voltage.
- **Vin** – It is the input voltage.
- **Analog Pins** – The pins numbered from A0 to A5 are analog pins. The function of Analog pins is to read the analog sensor used in the connection. It can also act as GPIO (General Purpose Input Output) pins.

2. Architecture of Breadboard

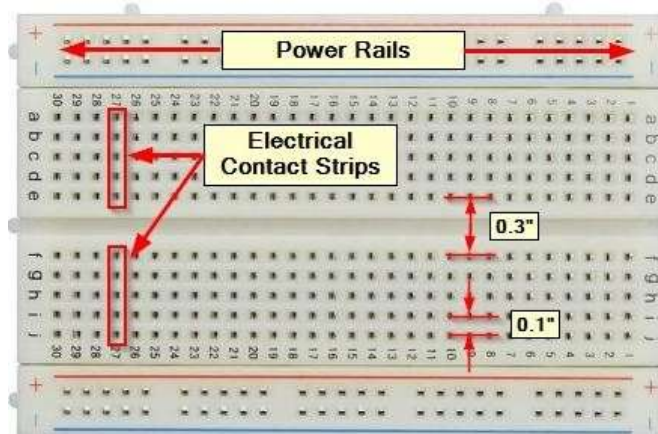


Figure 2.2

- **Layout:** Breadboard usually has a grid of holes with rows of metal clips underneath. The hole is usually divided into five holes with a break in the middle. This break allows you to easily connect things in between without causing a short circuit.

- **Connection points:** Each hole on the breadboard connects to another piece in its row or column with metal clips beneath the surface. The clips in the set of five are electrically connected to each other, but there is no connection between the set of five. This allows both horizontal and vertical connection.
- **Power Rails:** Most breadboards have two long side rails called power rails. Usually, one side is marked "+" and the other side is marked "-". These are used to power your circuit. You can connect a power source such as a battery or power supply to these screws.
- **Component placement:** Components such as resistors, LEDs, capacitors, and integrated circuits (ICs) are placed in holes in the rails of the breadboard. Then bend or cut pieces of the legs or make them fit snugly into the hole.
- **Connectivity:** You can use jumpers (usually made of flexible wire) to make connections between components by placing one end in one hole and the other end in another hole. This allows you to create circuits by connecting components together on a breadboard.
- **Testing and Prototyping:** After assembling the parts and making the necessary connections, you can test the circuit by applying power and observing its behavior. If something doesn't work as expected, you can easily modify the circuit by adjusting some things or changing connections.
- **Reusable:** One of the beautiful things about breadboards is that they can be reused. You can quickly remove electronics by removing parts and cables, allowing you to redo your design or use the breadboard for multiple projects.

Experiment No: 3

AIM: To study and install the Arduino IDE.

OBJECTIVE: Install and set up the Arduino Integrated Development Environment (IDE) to write, compile, and upload programs to Arduino boards.

The Arduino Software (IDE) is easy-to-use and is based on the Processing programming environment. The Arduino Integrated Development Environment (IDE) is a cross-platform application (for Windows, macOS, Linux) that is written in functions from C and C++. The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. This software can be used with any Arduino board.

The Arduino Software (IDE) – contains:

- A text editor for writing code
- A message area
- A text console
- A toolbar with buttons for common functions and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

Installation of Arduino Software (IDE):

Step 1: Go to the Arduino.cc Website

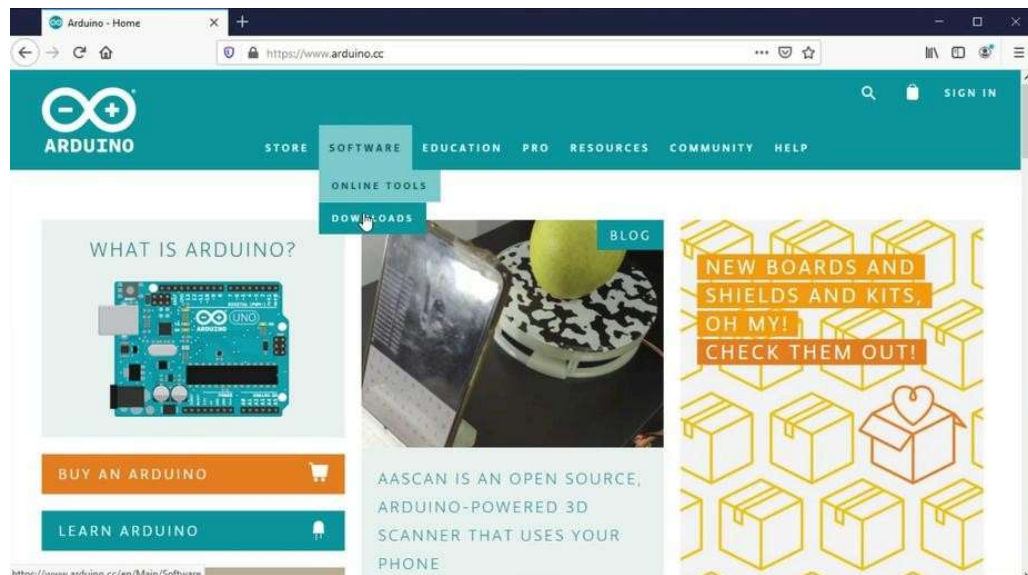


Figure 3.1

Go to the website www.arduino.cc in order to download the software.

Hover over the 'Software' tab and click on 'Downloads'.

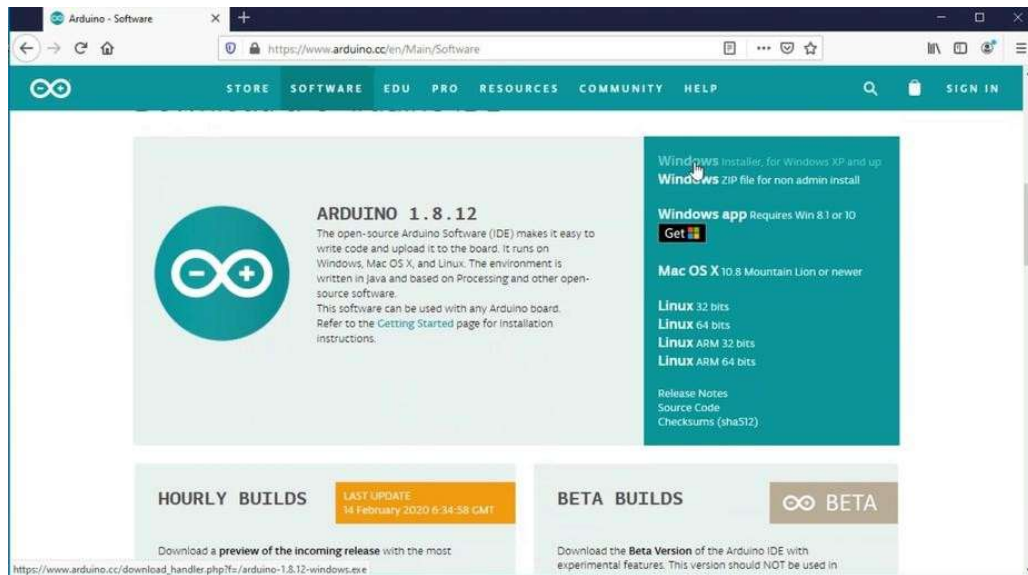
Step 2: Click on the Download Link

Figure 3.2

Scroll down until you see the link that says 'Windows installer' and click on it.

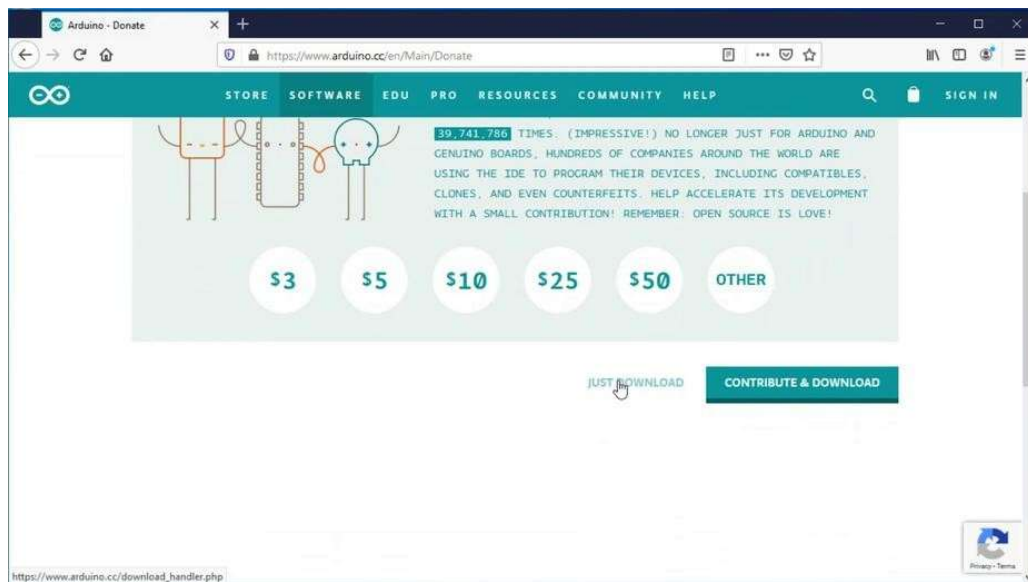
Step 3: Begin the Download

Figure 3.3

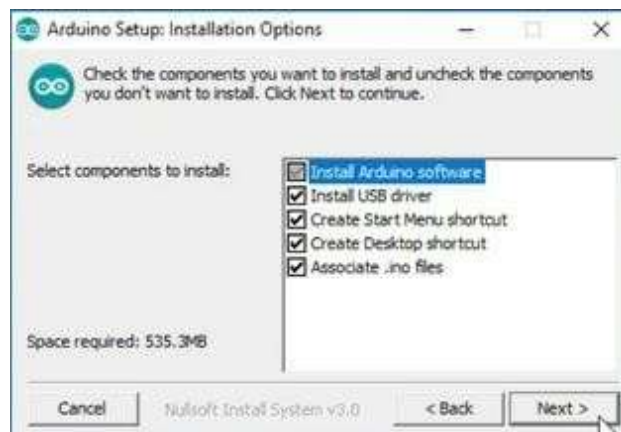
After clicking on the download link you'll be redirected to the donation page, here you can donate or skip it if you like by clicking on the 'Just download' link.

Step 4: Begin the Installation Process*Figure 3.4*

Open the downloaded file.

A new window will open asking you to agree to the license agreement.

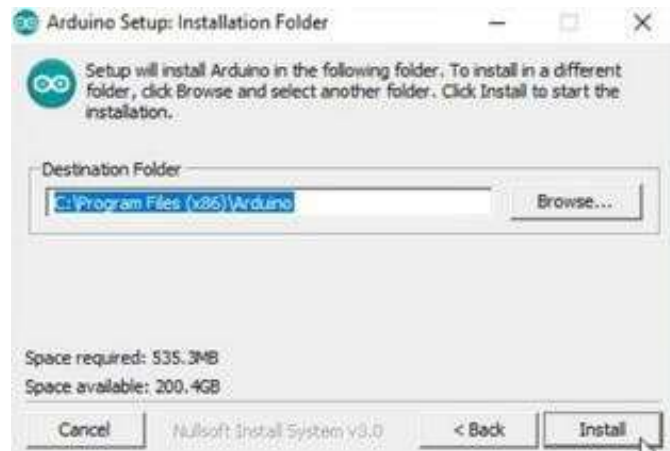
Click on 'I agree' to continue.

Step 5: Select What to Install*Figure 3.5*

Now you'll see all the available options to install the software with.

If you don't know what you need, it is best to keep everything checked as you can change it later when the installation has finished.

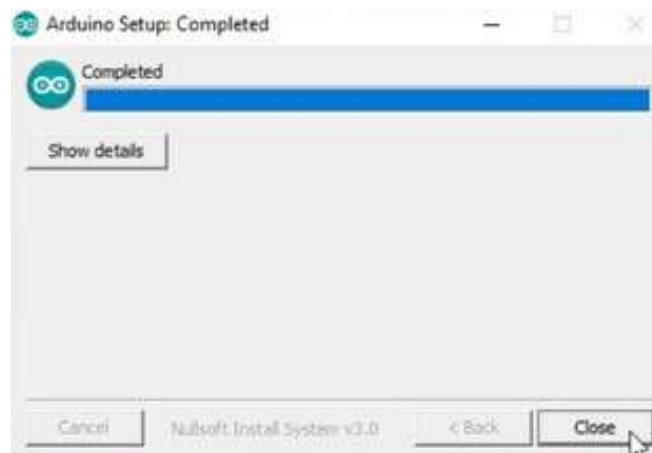
Click on 'Next' to continue.

Step 6: Choose the Installation Path*Figure 3.6*

Now you have to choose the path the software will be installed in.

it is fine to leave it at the configured location but if you want the Arduino IDE somewhere else installed you can change that here.

Click on 'Install' to begin the installation.

Step 7: Finish the Installation*Figure 3.7*

Wait until the installation is finished, it shouldn't take very long.

When the installation is finished you may click on 'close' to end the setup wizard.

Step 8: Launch the Arduino IDE*Figure 3.8*

Experiment No: 4

AIM: To learn how to write and upload a program in the Arduino IDE using various ports and boards.

OBJECTIVE: Understand the steps for selecting the correct board and port, writing a sketch, compiling the code, and uploading it to the Arduino UNO.

- **Install Arduino IDE:** If you haven't already, download and install the Arduino IDE from the official Arduino website (<https://www.arduino.cc/en/software>). Ensure that you choose the appropriate version for your operating system.
- **Open Arduino IDE:** Launch the Arduino IDE once it's installed on your computer.
- **Select Board:** Go to the "Tools" menu and select "Board." Choose the appropriate board you're using from the list. If your board is not listed, you may need to install the board's specific drivers or support package via the Arduino IDE's Board Manager.
- **Select Port:** In the same "Tools" menu, navigate to the "Port" submenu. Select the port to which your Arduino board is connected. If you're unsure which port to choose, you can check your operating system's device manager or system information for the connected Arduino board.
- **Write Code:** Start writing your Arduino program in the IDE's editor window. You can write code to control sensors, actuators, or perform any other tasks that your project requires. Arduino programming language is based on C/C++, so you can use familiar syntax and programming constructs.
- **Verify Code:** Before uploading your code to the Arduino board, click on the checkmark icon (or go to "Sketch" > "Verify/Compile") to verify that your code compiles without errors. This step ensures that there are no syntax errors in your code.
- **Upload Code:** Once your code is error-free, click on the right arrow icon (or go to "Sketch" > "Upload") to upload your code to the Arduino board. The Arduino IDE will compile your code and send it to the selected board via the chosen port.
- **Monitor Serial Output (Optional):** If your program includes serial communication, you can open the serial monitor in the Arduino IDE by clicking on the magnifying glass icon (or going to "Tools" > "Serial Monitor"). This allows you to view messages or data sent from the Arduino board and can be helpful for debugging.
- **Test and Debug:** After uploading your code, test your Arduino project to ensure that it behaves as expected. If you encounter any issues, you can use the serial monitor for debugging or modify your code accordingly.

- **Iterate and Refine:** Continue iterating your code, adjusting as needed to improve functionality or address any issues that arise during testing.

Experiment No: 5

AIM: To learn how to add libraries in the Arduino IDE and set up the environment for programming.

OBJECTIVE: Understand how to install libraries in the Arduino IDE and configure the software to support different sensors and modules.

Methods to add libraries to arduino ide

1. Using the Library Manager
 - Do one of the following:
 - In the menu bar, select Tools > Manage Libraries...
 - In the sidebar, click on the Library Manager icon button.
 - Filter the available libraries by typing something (such as a library name) in the text field above the listed libraries.
 - Find the library in the search results. The results are listed alphabetically, so you may need to scroll down the list.
 - Find a library you want to install. You can review the description and author. When you've found a library you want to install, click **Install**. The latest version is selected by default.

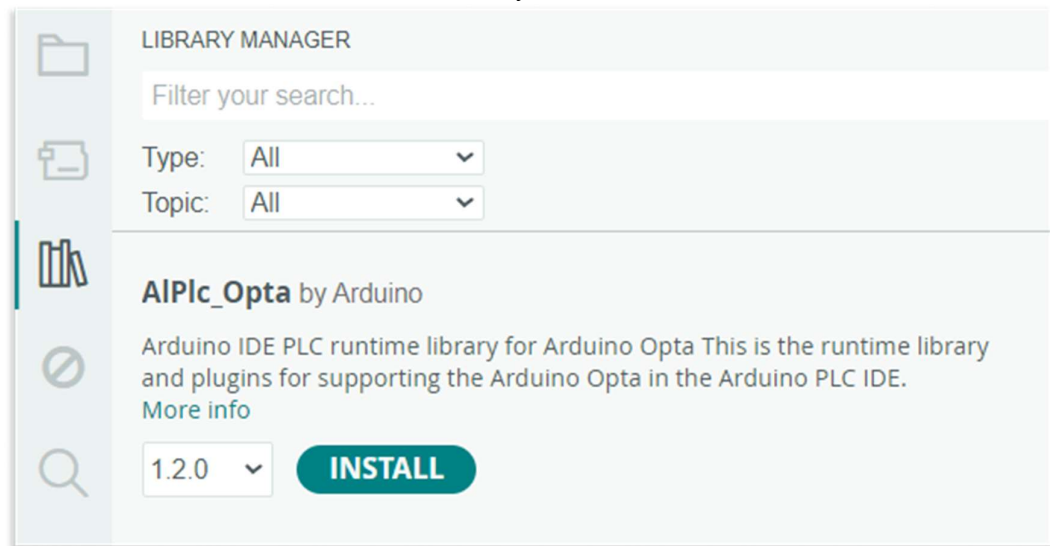


Figure 5.1

2. Importing a .zip library
 - In the menu bar, go to Sketch > Include Library > Add .ZIP Library...
 - You will be prompted to select the library you want to add. Navigate to the .zip file's location and open it.
3. Manual installation
 - Go to the directory where you have downloaded the .zip library file.
 - Extract the .zip file.
 - Select the main folder (it should have the library's name) and move it to the "libraries" folder inside your sketchbook. Restart Arduino IDE.

Experiment No: 6

AIM: To demonstrate how to use Autodesk TinkerCAD for simulating Arduino projects.

OBJECTIVE: Learn how to create, simulate, and test Arduino circuits virtually using the TinkerCAD platform.

Step 1: Requirements

- A computer or a laptop with a good internet connection.
- Internet browser (I used Google Chrome).
- Autodesk Tinkercad account

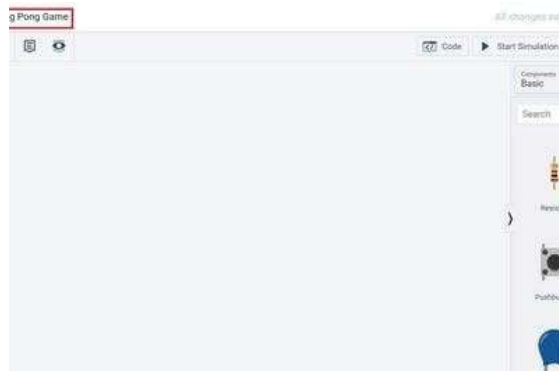


Figure 6.1

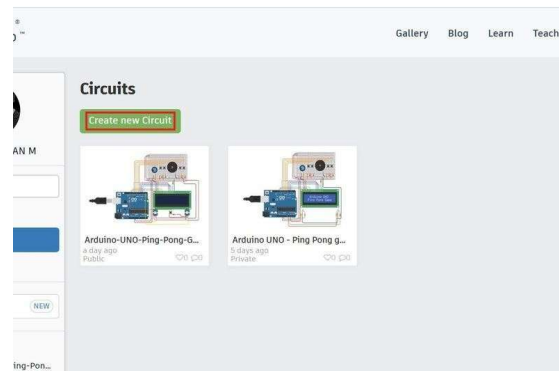


Figure 6.2

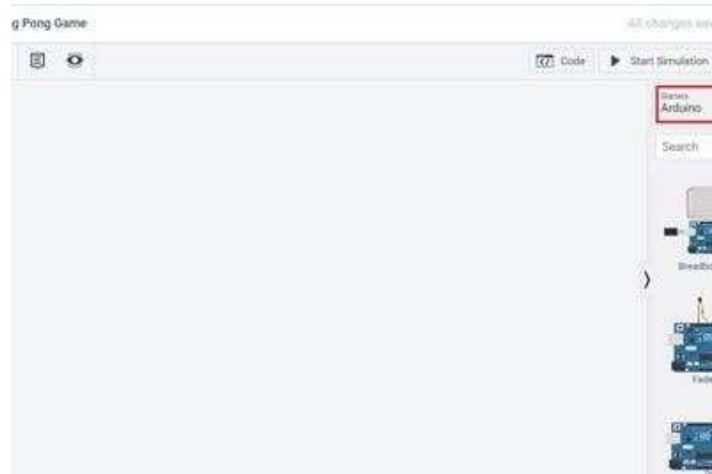


Figure 6.3

Step 2: Component Placement

- Open your browser.
- Enter the Autodesk Tinkercad website.
- Login to your Autodesk Tinkercad account.
- On the left of the website, you can find a circuit button, click on it and then click on create new circuit to create a new circuit.
- You'll be taken to the next page where you have to make the circuit connections and program the game.
- On the left top near the Autodesk Tinkercad logo, you can enter a new name for the project.
- Now on the right side of the webpage under the components tab, drag and drop the following components.
 - 1 x Arduino UNO board.
 - 2 x NeoPixel LEDs.
 - 1 x piezoelectric crystal.
 - 6 x resistors.
 - 5 x pushbuttons.
 - 1 x potentiometer.
 - 1 x LCD display 16x2.
 - 1 x breadboard.
- Make the circuit connection as per the following circuit diagram.

Step 3: Circuit Diagrams and Connections:

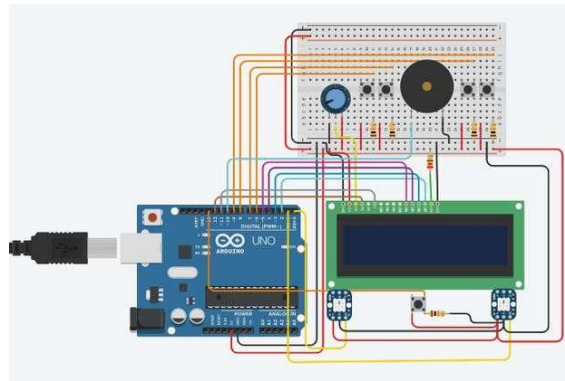


Figure 6.4

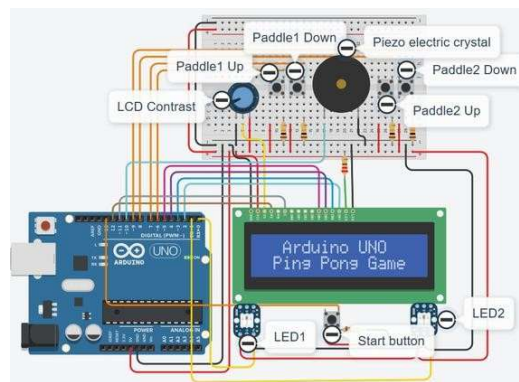


Figure 6.5

- Arduino UNO Connections:
 - Arduino UNO 0 -> NeoPixel LED1 in
 - Arduino UNO 1 -> NeoPixel LED2 in
 - Arduino UNO 2 -> LCD DB 7
 - Arduino UNO 3 -> LCD DB 6
 - Arduino UNO 4 -> LCD DB 5
 - Arduino UNO 5 -> LCD DB 4
 - Arduino UNO 6 -> Paddle1 Up pushbutton terminal 2 and 10K Ω pull down resistor
 - Arduino UNO 7 -> Paddle1 Down pushbutton terminal 2 and 10K Ω pull down resistor
 - Arduino UNO 8 -> Paddle2 Up pushbutton terminal 2 and 10K Ω pull down resistor
 - Arduino UNO 9 -> Paddle2 Down pushbutton terminal 2 and 10K Ω pull down resistor
 - Arduino UNO 10 -> piezoelectric crystal positive.
 - Arduino UNO 11 -> LCD Enable
 - Arduino UNO 12 -> LCD Register select
 - Arduino UNO 13 -> Start pushbutton terminal 2 and 10K Ω pull down resistor
 - Arduino UNO 5v -> LCD VCC, potentiometer terminal 2, NeoPixel LED1 + and NeoPixel LED2 +
 - Arduino UNO GND -> LCD GND, potentiometer terminal 1, NeoPixel LED1 G and NeoPixel LED2 G

- LCD connections:
 - Contrast -> potentiometer wiper
 - LCD LED Cathode -> 220Ω pullup resistor
 - LCD LED Anode -> Arduino UNO GND
- Pushbuttons:
 - Connect all pushbutton's terminal 1 to Arduino UNO 5v.

Step 4: Coding:

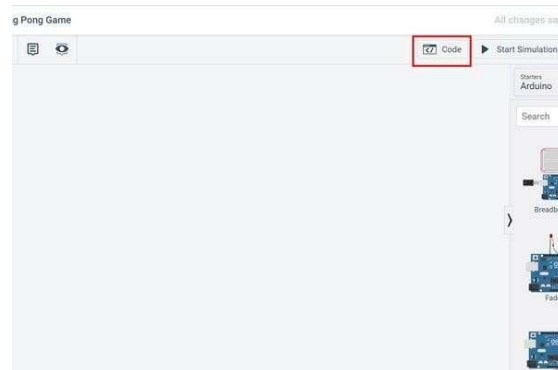


Figure 6.6

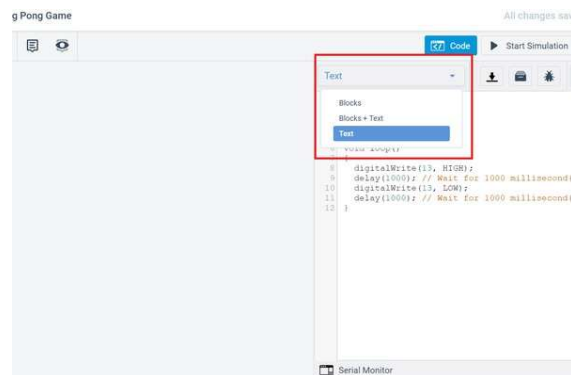


Figure 6.7

- Now you have to code the Arduino UNO board.
- One the top right of the website, we can see a code button, click on it.
- Select text under the drop-down box.
- Now copy and paste the code in the text box from any one of the following links.

GitHub

At this time we have completed the connections and coding part and the project is ready to simulate.

Step 5: Simulation:

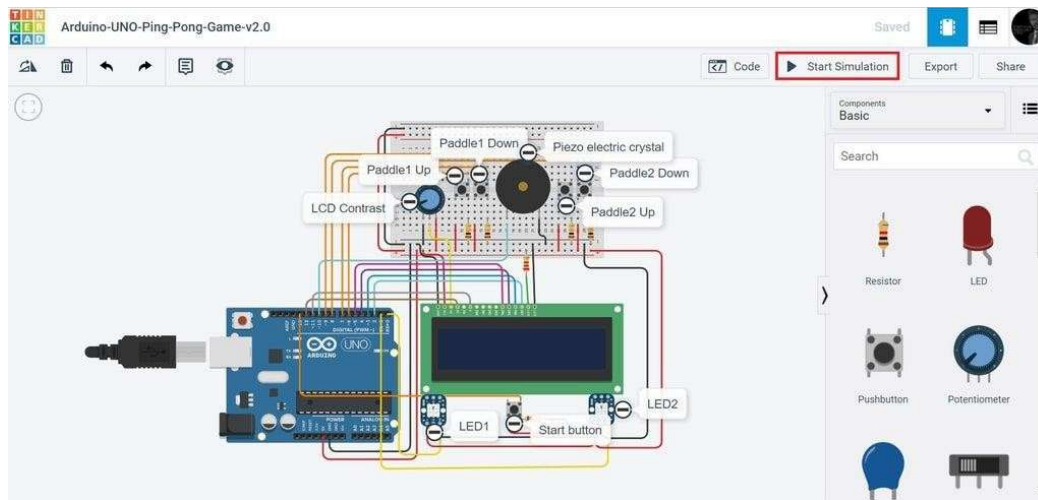


Figure 6.8

- To start the simulation, click on the simulation button on the top right of the website.
- Adjust the potentiometer until you get a clear view of the game on the LCD display.
- Use the start button to start the game and paddle1 UP, paddle 2 Down, paddle2 Up and paddle2 Down pushbuttons to control the paddle1 and paddle2.

Experiment No: 7

AIM: To write a program to blink an LED using a battery and a resistor.

OBJECTIVE: Understand basic circuit connection and how to turn an LED on/off in intervals using simple hardware.

Components Required:

- 1 x Battery
- 1 x Arduino Uno
- 1 x resistor
- 1 x Led

Procedure:

- Connect the positive leg of the LED (longer leg) to the resistor.
- Connect the other end of the resistor to the digital pin 13 on the Arduino board.
- Connect the negative leg of the LED (shorter leg) to the GND (ground) pin on the Arduino board.

Circuit Diagram:

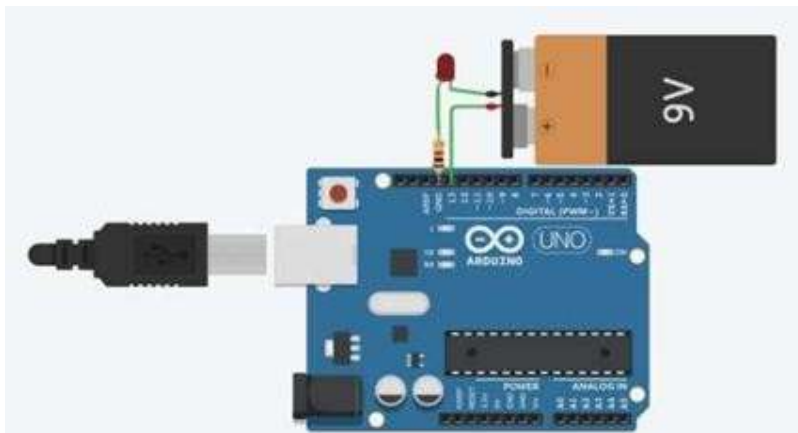


Figure 7.1

Code:

```
// Define the pin to which the LED is connected
const int ledPin = 13; // Assuming the LED is connected to digital pin 13

void setup() {
    // Initialize the digital pin as an output
    pinMode(ledPin, OUTPUT);
}

void loop() {
```

```
// Turn the LED on (HIGH)
digitalWrite(ledPin, HIGH);
delay(1000); // Wait for 1 second
// Turn the LED off (LOW)
digitalWrite(ledPin, LOW);
delay(1000); // Wait for 1 second
}
```

Output:

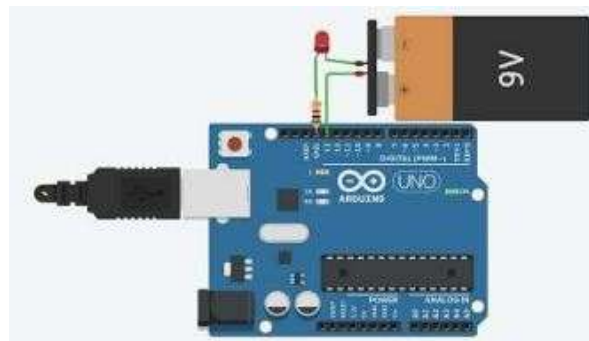


Figure 7.2

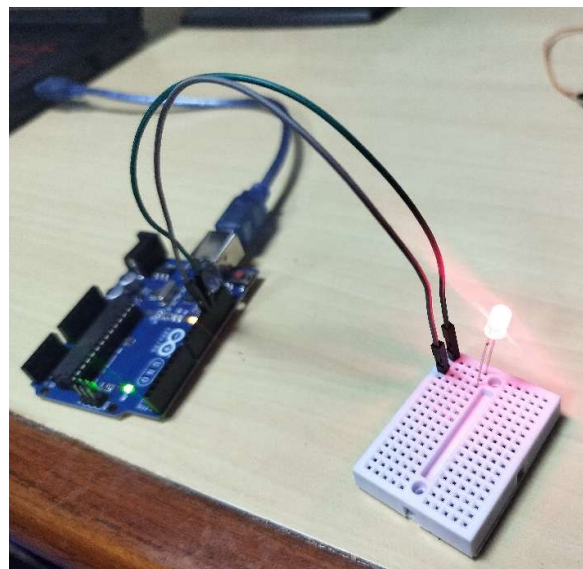


Figure 7.3

Experiment No: 8

AIM: To write a program to blink multiple LEDs using Arduino UNO.

OBJECTIVE: Learn how to control multiple LEDs through programming in two ways:

- a) Without using a breadboard
- b) With using a breadboard

A. Without using breadboard

Components Required:

- 1 x USB Cable
- 1 x Arduino Uno
- 1 x BreadBoard
- 3 x LED
- 3 x Resistor
- 4 x Jumper Wires

Procedure:

- Connect the positive leg of the first LED to digital pin 13 on the Arduino board.
- Connect the negative leg of the first LED to the GND (ground) pin on the Arduino board.
- Connect the positive leg of the second LED to digital pin 12 on the Arduino board.
- Connect the negative leg of the second LED to the GND (ground) pin on the Arduino board.
- Continue this process for all the LEDs, making sure to use a different digital pin for each LED.

Circuit Diagram:

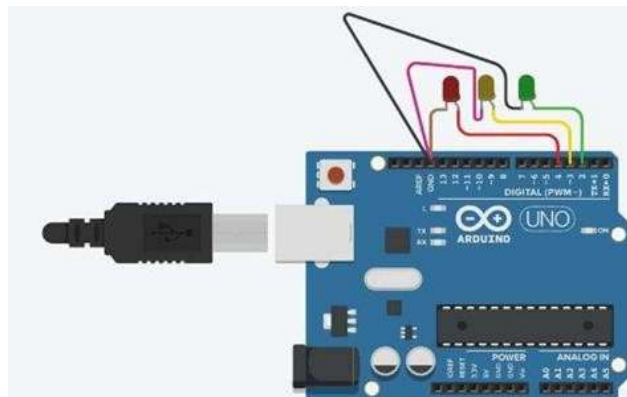


Figure 8.1

Code:

```
// Define the pins to which the LEDs are connected
```

```
const int ledPin1 = 2; // Connect LED 1 to digital pin 2
const int ledPin2 = 3; // Connect LED 2 to digital pin 3
const int ledPin3 = 4; // Connect LED 3 to digital pin 4
void setup() {
  // Initialize the digital pins connected to LEDs as outputs
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(ledPin3, OUTPUT);}
void loop() {
  // Turn LED 1 on
  digitalWrite(ledPin1, HIGH);
  delay(500); // Wait for 500 milliseconds (half a second)
  // Turn LED 1 off
  digitalWrite(ledPin1, LOW);
  // Turn LED 2 on
  digitalWrite(ledPin2, HIGH);
  delay(500); // Wait for 500 milliseconds (half a second)
  // Turn LED 2 off
  digitalWrite(ledPin2, LOW);
  // Turn LED 3 on
  digitalWrite(ledPin3, HIGH);
  delay(500); // Wait for 500 milliseconds (half a second)
  // Turn LED 3 off
  digitalWrite(ledPin3, LOW);}
```


Output:

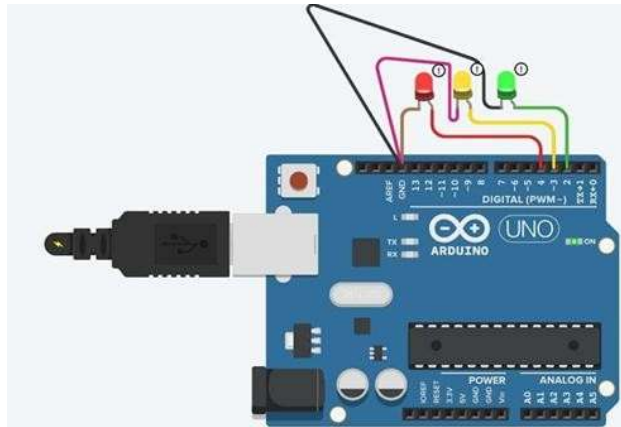


Figure 8.2

B. With breadboard

Components required:

- 1 x Arduino board
- 3 x LEDs
- Resistors (220-1 Ohms)
- 1 x Breadboard
- 4 x Jumper wires

Procedure:

- Connect the positive leg of the first LED (longer leg) to a resistor.
- Connect the other end of the resistor to a row on the breadboard.
- Connect a jumper wire from the same row on the breadboard to a digital pin on the Arduino board (e.g. digital pin 13).
- Connect the negative leg of the first LED (shorter leg) to the GND (ground) pin on the Arduino board.
- Repeat steps 1-4 for each additional LED, using a different digital pin for each one.

Circuit Diagram:

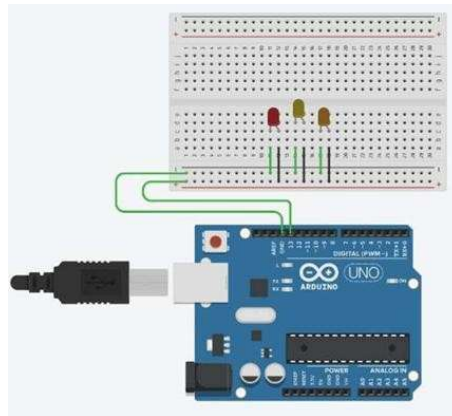


Figure 8.3

Output:

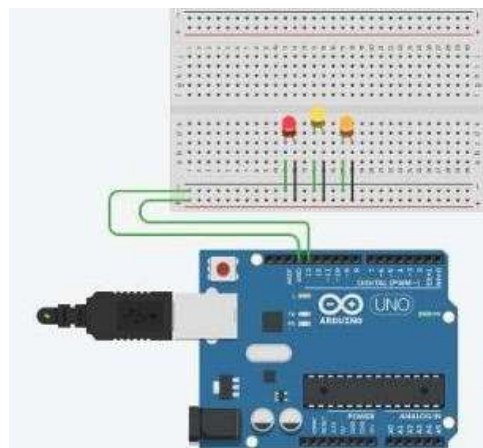


Figure 8.4

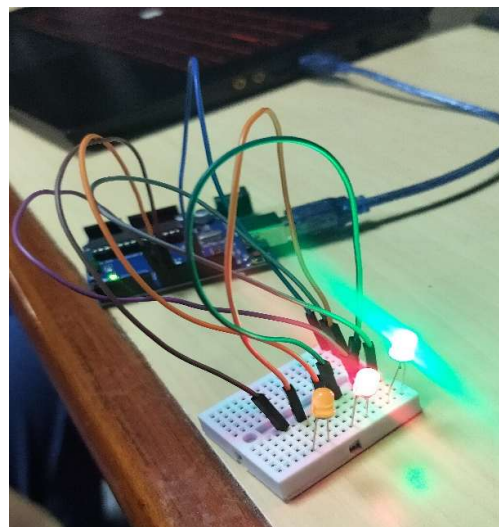


Figure 8.5

Experiment No: 9

AIM: To write a program to fade an LED connected to pin 9 using the `analogWrite()` function.

OBJECTIVE: Understand how Pulse Width Modulation (PWM) works and how to use it to control LED brightness.

Components Required:

- 1 x USB Cable
- 1 x Arduino Uno
- 1 x BreadBoard
- 1 x LED
- 3 x Resistor
- 4 x Hook-up Wires

Procedure:

- Connect the anode (the longer, positive leg) of your LED to digital output pin 9 on your board through a 220 ohm resistor. Connect the cathode (the shorter, negative leg) directly to ground.
- Declare pin 9 as the `ledPin` and set it as an output in the `setup()` function.
- Create a for loop that gradually increases the brightness of the LED from 0 to 255, with a step of 1.
- Inside the for loop, use the `analogWrite()` function to write the current brightness value to the `ledPin`.
- Add a delay of 15 milliseconds using the `delay()` function to slow down the fading effect.
- Create another for loop that gradually decreases the brightness of the LED from 255 to 0, with a step of 1.
- Inside the second for loop, use the `analogWrite()` function to write the current brightness value to the `ledPin`.
- Add a delay of 15 milliseconds using the `delay()` function to slow down the fading effect.
- Repeat steps 3-8 to create a smooth fading effect.

Circuit Diagram:

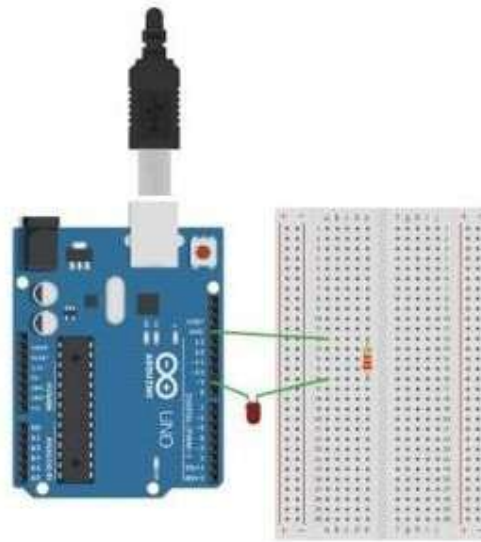


Figure 9.1

Code:

```
int ledPin = 9;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  for(int ledVal = 0; ledVal <= 255; ledVal +=1) {
    analogWrite(ledPin, ledVal);
    delay(15);
  }
  for(int ledVal = 255; ledVal >= 0; ledVal -=1) {
    analogWrite(ledPin, ledVal);
    delay(15);
  }
}
```

Output:

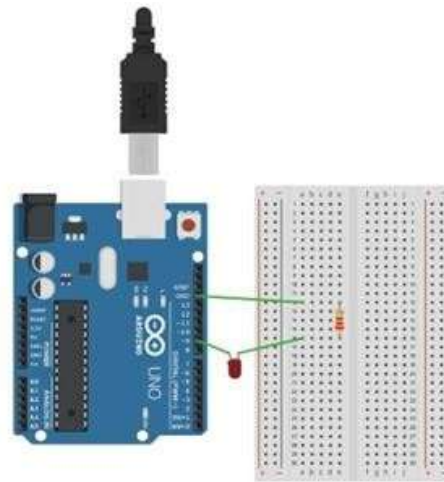


Figure 9.2

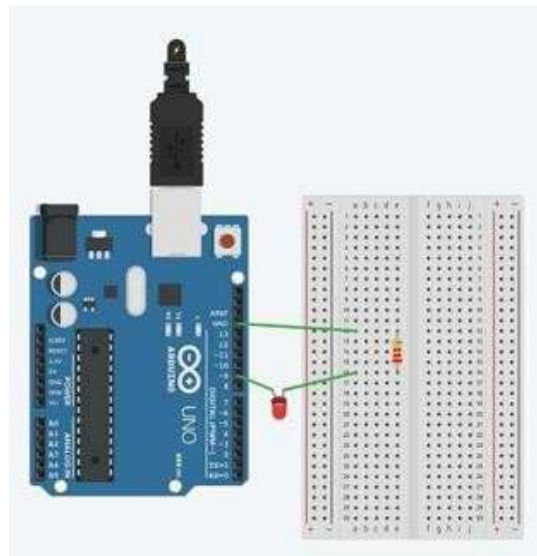


Figure 9.3

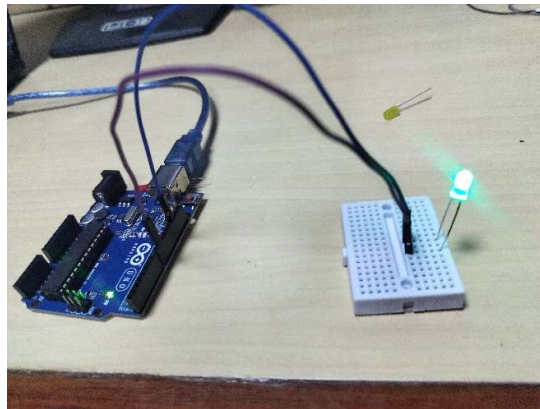


Figure 9.4

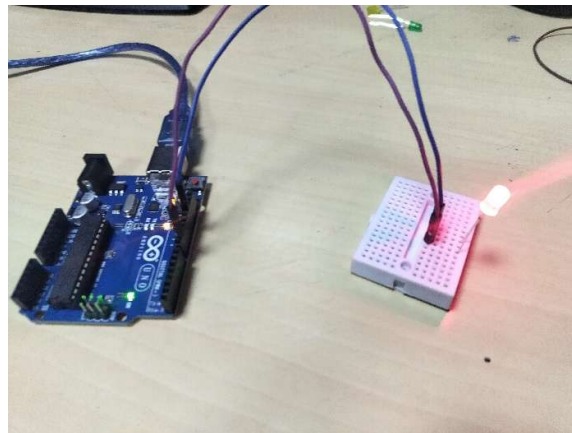


Figure 9.5

Experiment No: 10

AIM: To interface an LDR sensor with an LED using Arduino UNO.

OBJECTIVE:

- a) Turn LED ON/OFF based on ambient light using LDR
- b) Adjust LED brightness based on LDR sensor readings

A. Turn LED ON/OFF based on ambient light using LDR

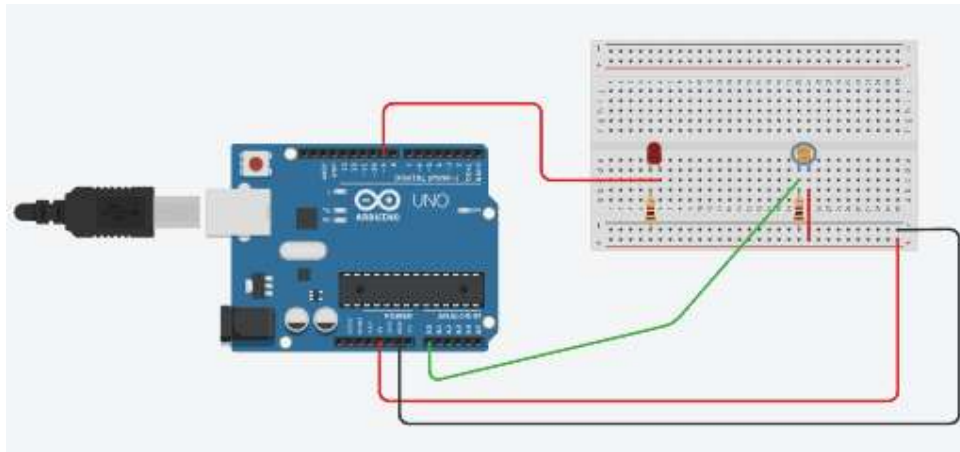
Components Required:

- 1 x Arduino Uno
- 1 x USB Cable
- 1 x Breadboard
- 1 x LDR (Light Dependent Resistor)
- 1 x 10k Ω Resistor
- 1 x LED
- 1 x 220 Ω Resistor
- Jumper Wires

Procedure:

1. Connect the LDR:
 - Place the LDR on the breadboard.
 - Connect one leg of the LDR to 5V on the Arduino.
 - Connect the other leg of the LDR to Analog pin A0.
 - Place a 10k Ω resistor between A0 and GND to make a voltage divider.
2. Connect the LED:
 - Place the LED on the breadboard.
 - Connect the longer leg (anode) of the LED to digital pin 13 on the Arduino through a 220 Ω resistor.
 - Connect the shorter leg (cathode) directly to GND.
3. Upload the Code:
 - Open the Arduino IDE.
 - Select the correct board and port from the Tools menu.
 - Paste the code into a new sketch.
 - Upload the code to the Arduino.
4. Test the Setup:
 - Open the Serial Monitor to see the LDR readings.
 - Cover the LDR — the LED should turn ON.
 - Shine light on the LDR — the LED should turn OFF.

Circuit Diagram:

*Figure 10.1*

Code:

```
int sensorPin = A0;
int sensorValue = 0;
int led = 9;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop(){
  sensorValue = analogRead(sensorPin);
  Serial.println(sensorValue);
  if(sensorValue < 100){
    Serial.println("LED light on");
    digitalWrite(led,HIGH);
    delay(1000);
  }
  digitalWrite(led,LOW);
  delay(sensorValue);}
```


Output:

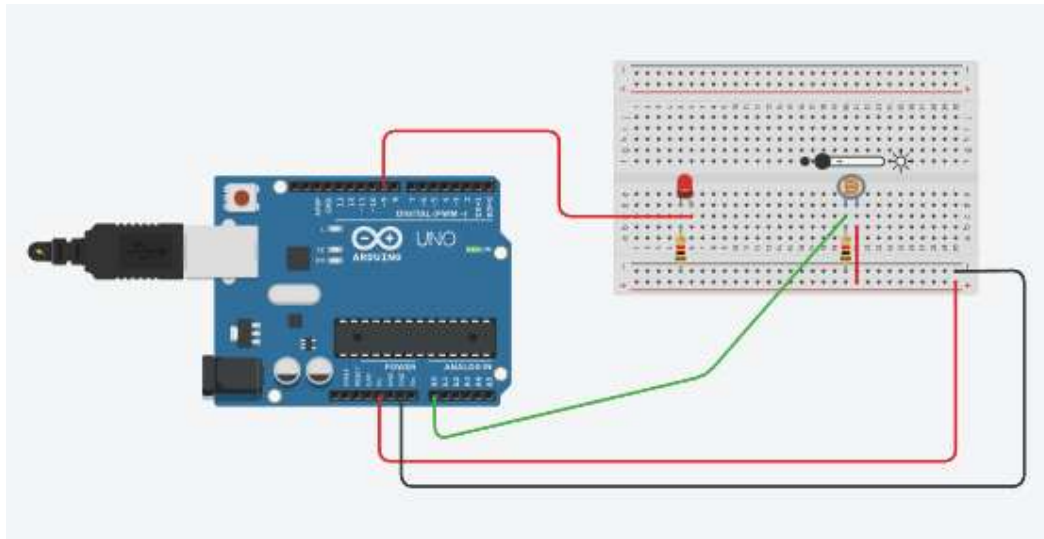


Figure 10.2

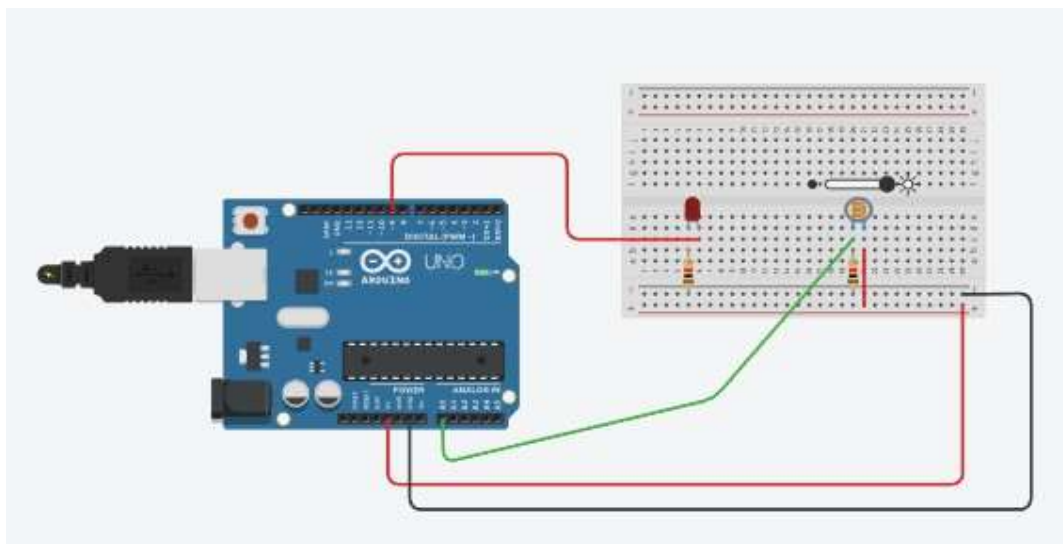


Figure 10.3

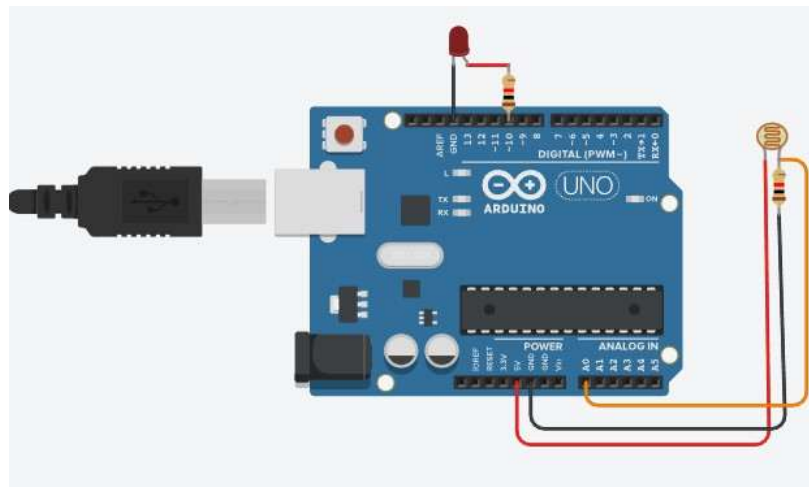
B. Adjust LED brightness based on LDR sensor readings

Components Required:

- 1 x Arduino Uno
- 1 x USB Cable
- 1 x LDR (Light Dependent Resistor)
- 1 x 10k Ω Resistor
- 1 x LED
- 1 x 220 Ω Resistor
- Jumper Wires

Procedure:

1. Connect the LDR:
 - Connect one leg of the LDR to 5V on the Arduino.
 - Connect the other leg of the LDR to Analog pin A0.
 - Place a 10k Ω resistor between A0 and GND to create a voltage divider.
2. Connect the LED:
 - Connect the longer leg (anode) of the LED to digital pin 13 through a 220 Ω resistor.
 - Connect the shorter leg (cathode) of the LED to GND.
3. Upload the Code:
 - Open the Arduino IDE.
 - Select the correct board and COM port from the Tools menu.
 - Copy and paste the following code into a new sketch.
 - Upload it to the Arduino.
4. Test the Setup:
 - Open the Serial Monitor (set baud rate to 9600).
 - Cover the LDR to simulate darkness — the LED should turn ON.
 - Shine light on the LDR — the LED should turn OFF.

Circuit Diagram:*Figure 10.4***Code:**

```
int sensor = A0;
int lightvalue=0;
int val =0;
int led = 10;
void setup() {
  Serial.begin(9600);
```

```
pinMode(sensor,INPUT);  
}  
void loop() {  
    lightvalue = analogRead(sensor);  
    //for this LDR , high value is 679 and low is 6  
    lightvalue = map(lightvalue,6,679,255,0);  
    Serial.println(lightvalue);  
    val= lightvalue;  
    //val = lightvalue/2.7;  
    //val = 255-val;  
    //Serial.println(val);  
    analogWrite(led,val);  
}
```

Output:

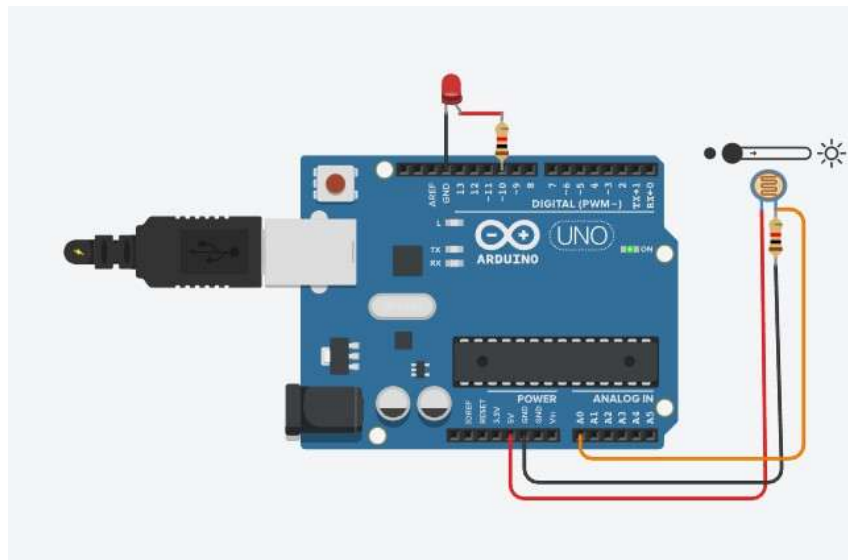
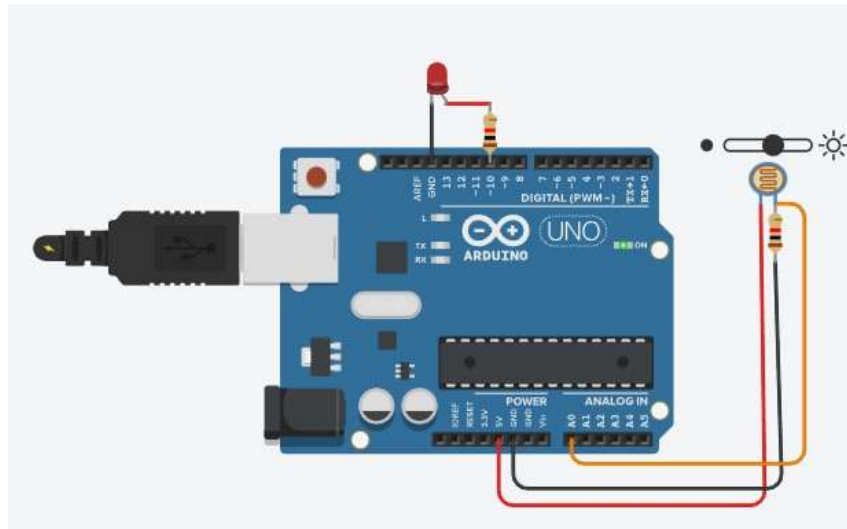
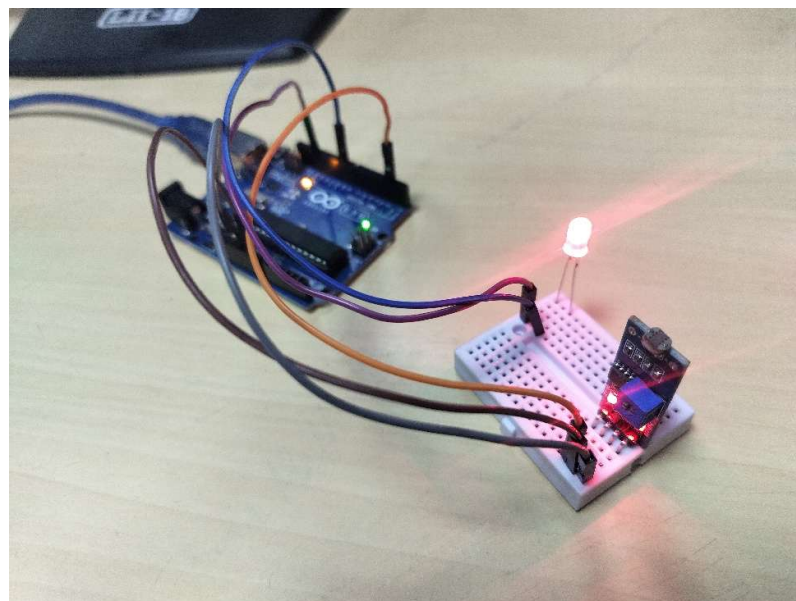


Figure 10.5

*Figure 10.6**Figure 10.7*

Experiment No: 11

AIM: To write a program to measure the distance of an object using an Ultrasonic Sensor (HC-SR04) with Arduino UNO.

OBJECTIVE: Understand how ultrasonic sensors measure distance and learn to interface them with Arduino UNO for object detection.

Components Required:

- 1 × Arduino Uno
- 1 × HC-SR04 Ultrasonic Sensor
- 1 × 16x2 LCD Display (with I2C module or without, depending on wiring)
- 1 × Buzzer
- 1 × 220Ω Resistor (for buzzer current limiting)
- 1 × Breadboard
- Jumper Wires
- 1 × USB Cable

Wiring Instructions:

1. Ultrasonic Sensor (HC-SR04):
 - VCC → 5V on Arduino
 - GND → GND on Arduino
 - Trig → Digital Pin 9
 - Echo → Digital Pin 8
2. 16x2 LCD Display (Parallel mode, no I2C):
 - RS → Digital Pin 12
 - EN → Digital Pin 11
 - D4 to D7 → Digital Pins 5, 4, 3, 2
 - VSS & RW → GND
 - VDD → 5V
 - Vo → Connect to middle leg of a potentiometer (for contrast control)
 - A (Anode for backlight) → 5V through resistor (optional)
 - K (Cathode) → GND
3. Buzzer:
 - Positive leg → Digital Pin 7
 - Negative leg → GND
 - Add a 220Ω resistor in series with the buzzer (optional for volume control)
4. Powering Arduino:
 - Plug the Arduino Uno into your PC using a USB cable.

Working Principle:

- The ultrasonic sensor measures distance by sending sound waves and measuring their reflection.
- The LCD displays the measured distance in centimeters (cm).
- When the object is too close (e.g., less than 10 cm), the buzzer will beep to alert.

Circuit Diagram:

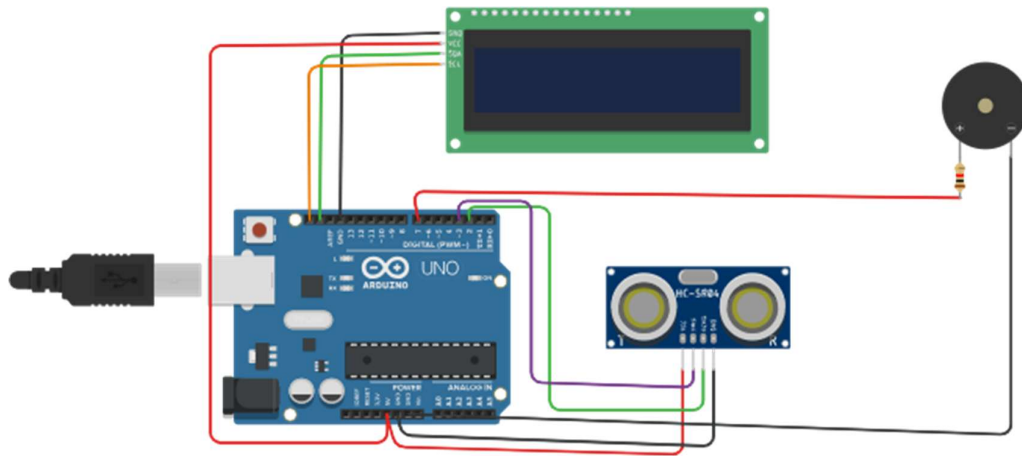


Figure 11.1

Code:

```
#include <LiquidCrystal_I2C.h> //required library
LiquidCrystal_I2C lcd(0x20, 16, 2); //(Address,Column,Row)
#define echoPin 2 // Connect echoPin to D2 pin of arduino
#define trigPin 3 // Connect trigPin to D3 pin of arduino
long duration; // Declare variable to store echo time duration
int distance; // Declare variable to store the result (distance)
void setup()
{
    lcd.init(); // initialize the lcd
    lcd.backlight(); // Turn on the Backlight

    pinMode(trigPin,OUTPUT); // Set the trigPin as an OUTPUT
    pinMode(echoPin, INPUT); // Set the echoPin as an INPUT
    // Serial Communication is starting with 9600 of baudrate speed
    Serial.begin(9600);

    // The text to be printed in serial monitor
    Serial.println("Distance measurement :");
```

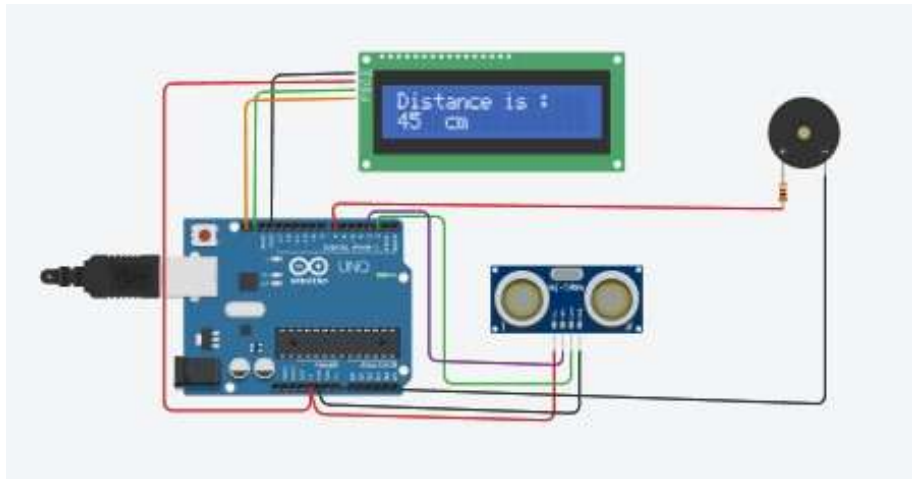
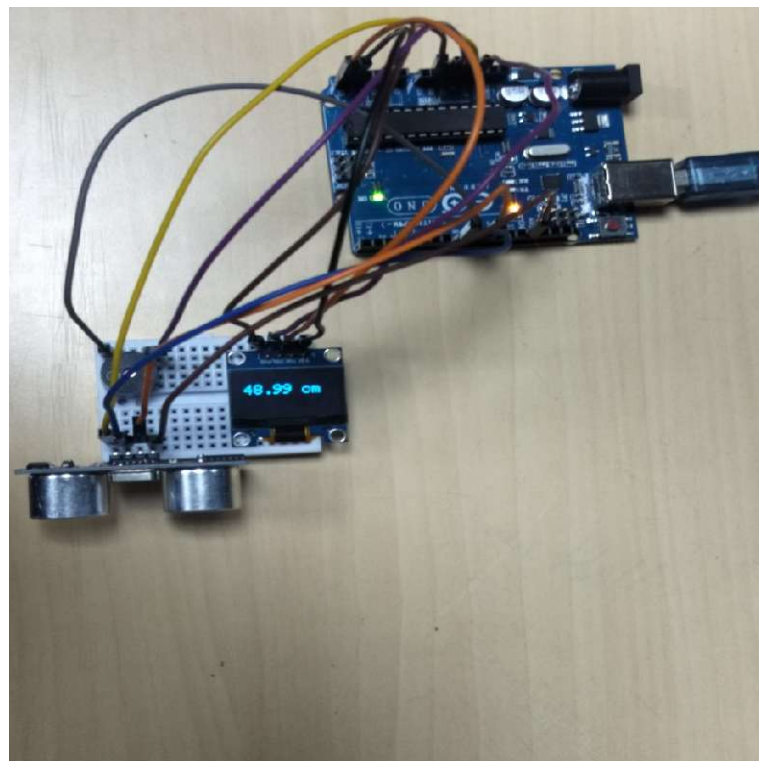
```
        delay(500); // delay between the print of the message and distance count
    }
    void loop()
    {
        digitalWrite(trigPin, LOW); //sets the trigPin to low initially
        delayMicroseconds(2); //waits for 2uS
        digitalWrite(trigPin, HIGH); //sets the trigPin to high
        delayMicroseconds(10); //waits for 10uS
        digitalWrite(trigPin, LOW); //sets the trigPin to low again
        duration = pulseIn(echoPin, HIGH); //records the time for which the echoPin is set high
        distance = duration * 0.0344 / 2; //estimates the object distance based on the duration
        Serial.print("Distance: ");
        Serial.print(distance);
        Serial.println(" cm");
        lcd.clear(); // Clear the display buffer
        lcd.setCursor(0, 0); // Set cursor for "Distance is:" (Column, Row)
        lcd.print("Distance is:"); // print "Distance is:" at (0, 0)
        lcd.setCursor(0,1); // Set cursor for output value (0, 1)

        lcd.print(distance); // print Output in cm at (0, 1)
        lcd.setCursor(4,1); // move cursor to (4, 1)
        lcd.print("cm"); // print "cm" at (4, 1)
        delay(1000);
        if(distance<90||distance>300){ //fixes the distance range
            delayMicroseconds(1);
            tone(7,400,100); //sets the buzzer to high
        }
        else{
            noTone(7); //sets the buzzer to low
        }
    }
}
```

}

}

Output:

*Figure 11.2**Figure 11.3*

Experiment No: 12

AIM: To control an LED using Arduino UNO with various functionalities.

OBJECTIVE: Write code for:

- a) Blinking an LED
- b) Fading an LED
- c) Dancing LED effect
- d) Controlling LED with a Button

A. Blinking an LED

Components Required:

- 1 × Arduino Uno
- 1 × LED (any color)
- 1 × 220Ω Resistor
- 1 × Breadboard
- 3 × Jumper Wires
- 1 × USB Cable (to upload code from PC)

Procedure:

1. Connect the LED:
 - Place the LED on the breadboard.
 - Connect the longer leg (anode) of the LED to digital pin 7 on the Arduino through a 220Ω resistor.
 - Connect the shorter leg (cathode) to GND on the Arduino.
2. Powering the Arduino:
 - Connect the Arduino to your computer using the USB cable.
3. Code to Blink LED
4. Testing the setup
 - Open Arduino IDE.
 - Paste the code above.
 - Go to Tools > Board > Arduino Uno.
 - Select the correct COM port under Tools > Port.
 - Click the Upload button (right arrow icon).
 - The LED should start blinking ON and OFF every 1 second.

Circuit Diagram:

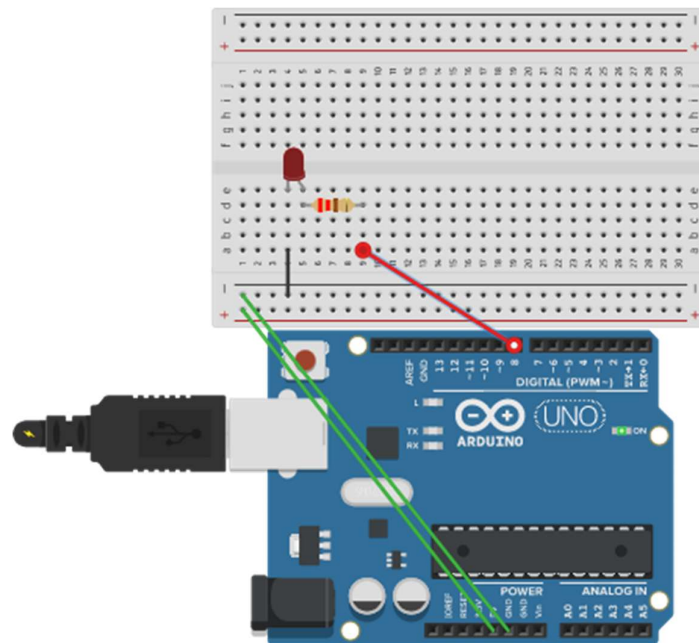


Figure 12.1

Code:

```
void setup() {  
  pinMode(7, OUTPUT); // Set pin 7 as an output  
}  
  
void loop() {  
  digitalWrite(7, HIGH); // Turn LED ON  
  delay(1000);           // Wait 1 second  
  digitalWrite(7, LOW);  // Turn LED OFF  
  delay(1000);           // Wait 1 second  
}
```

Output:

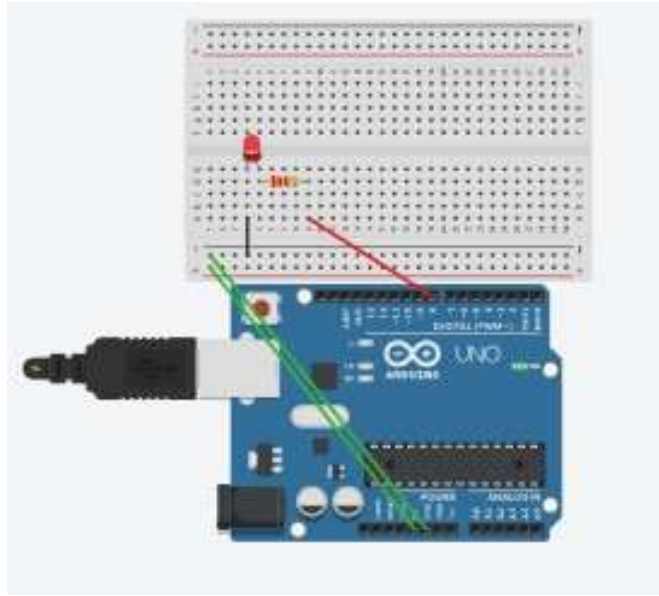


Figure 12.2

B. Fading an LED

Components Required:

- 1 × Arduino Uno
- 1 × LED (any color)
- 1 × 220Ω Resistor
- 1 × Breadboard
- 3 × Jumper Wires
- 1 × USB Cable (to upload code)

Circuit Connections:

1. LED:

- Place the LED on the breadboard.
- Connect the longer leg (anode) of the LED to pin 9 on Arduino.
- Connect a 220Ω resistor from the shorter leg (cathode) to GND on the Arduino.

2. Power:

- Connect the Arduino to your computer using the USB cable.

Note: Use pin 9 because it supports PWM (Pulse Width Modulation), which is needed for fading.

3. Code to Fading Led

4. Testing Setup

- Open Arduino IDE.
- Paste the code above.
- Go to Tools > Board > Arduino Uno.
- Select the correct Port.

- Click the Upload button.
- Watch the LED gradually fade in and out.

Circuit Diagram:

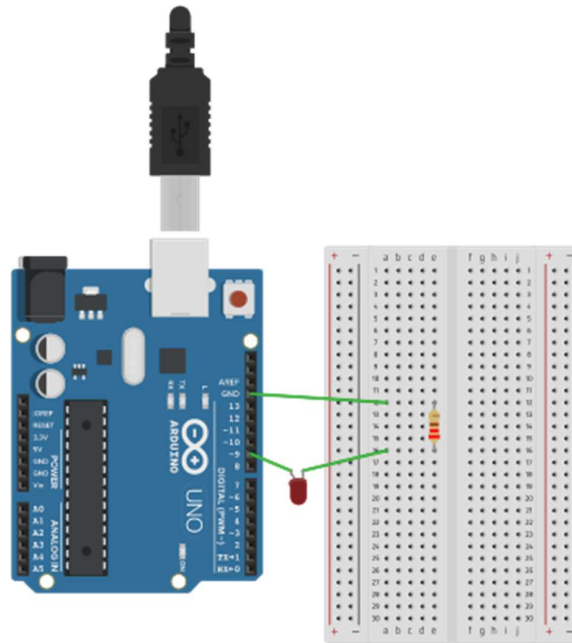


Figure 12.3

Code:

```
int le_fokin_lamp = 9;
int le_fokin_fade_bru = 0;

void setup() {
}

void loop() {

for (le_fokin_fade_bru = 0; le_fokin_fade_bru <= 255; le_fokin_fade_bru += 1) {
analogWrite(le_fokin_lamp, le_fokin_fade_bru); delay(5); }

    delay(1000);

for (le_fokin_fade_bru = 255; le_fokin_fade_bru >= 0; le_fokin_fade_bru -= 1) {
analogWrite(le_fokin_lamp, le_fokin_fade_bru); delay(5); }

    delay(1000);

for (le_fokin_fade_bru = 0; le_fokin_fade_bru <= 255; le_fokin_fade_bru += 1) {
analogWrite(le_fokin_lamp, le_fokin_fade_bru); delay(5); }

    delay(1000);
```

```
for (le_fokin_fade_bru = 255; le_fokin_fade_bru >= 0; le_fokin_fade_bru -= 1) {  
  analogWrite(le_fokin_lamp, le_fokin_fade_bru); delay(5); }  
  
  delay(5000);  
}
```

Output:

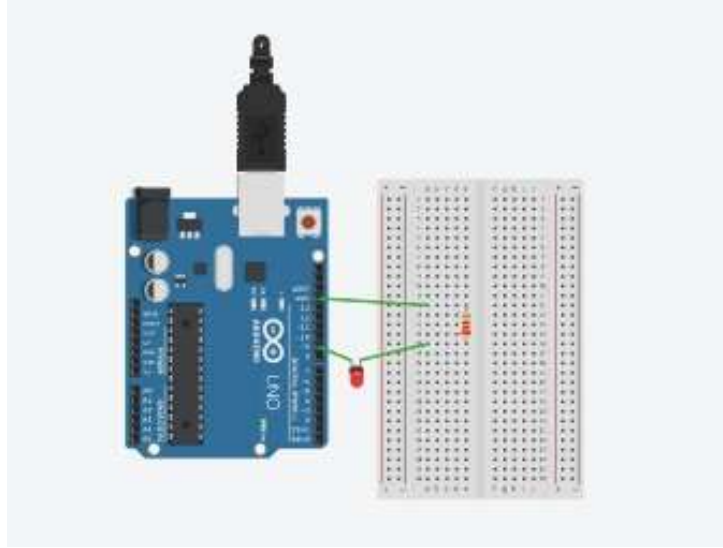


Figure 12.4

C. Dancing LED effect

Components Required:

- 1 × Arduino Uno
- 3 × Red LEDs
- 2 × Yellow LEDs
- 1 × Green LED
- 6 × 220Ω Resistors
- 1 × Breadboard
- Jumper Wires
- 1 × USB Cable

Wiring Instructions:

1. LED Connections:

- Connect each LED's longer leg (anode) to Arduino digital pins:
 - Red LEDs to pins 2, 3, 4
 - Yellow LEDs to pins 5, 6
 - Green LED to pin 7
- Connect a 220Ω resistor to the shorter leg (cathode) of each LED, then to GND rail on the breadboard.

2. Powering Arduino:

- Plug the Arduino Uno into your PC using a USB cable.

3. GND Wire:

- Connect Arduino GND to the breadboard's GND rail.

4. Code to Sequential LED Blinking

5. Testing the setup

- Open the Arduino IDE.
- Copy and paste the code above.
- Select Board: Arduino Uno and the correct Port under the Tools menu.
- Click Upload.
- LEDs will blink one by one in sequence like a wave or traffic signal effect.

Circuit Diagram:

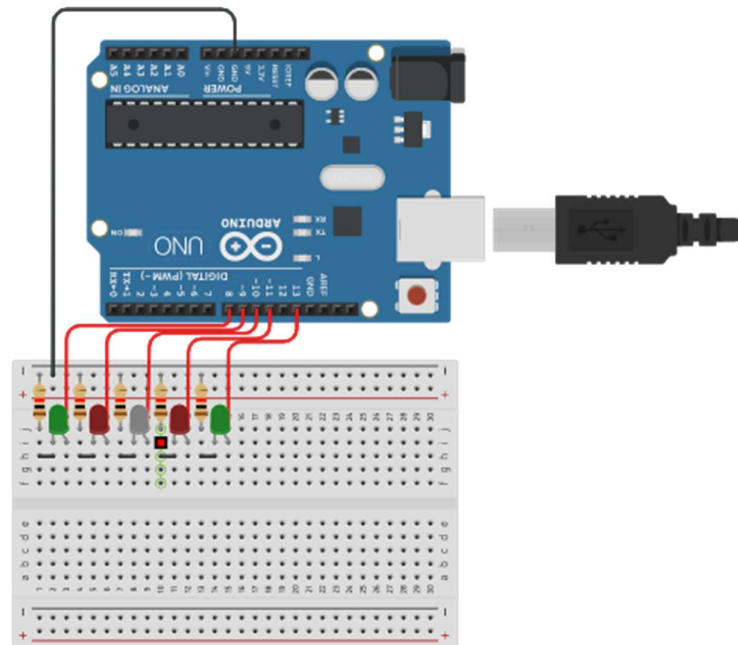


Figure 12.5

Code:

```
int pin1 = 8;
int pin2 = 9;
int pin3 = 10;
int pin4 = 11;
int pin5 = 13;
int pines[] = {pin1, pin2, pin3, pin4, pin5};
void blink(int pin, int mSeg){
digitalWrite(pin, HIGH);
delay(mSeg);
```

```
    digitalWrite(pin, LOW);
    delay(mSeg);
}

void onAll(int pines[], int nPines){
    for (int i = 0 ; i < nPines ; i++){
        digitalWrite(pines[i], HIGH);
    }
}

void offAll(int pines[], int nPines){
    for (int i = 0 ; i < nPines ; i++){
        digitalWrite(pines[i], LOW);
    }
}

void blinkAll(int pines[], int nPines, int mSeg){
    onAll(pines, nPines);
    delay(mSeg);
    offAll(pines, nPines);
    delay(mSeg);
}

void turnAround(int iterations, int mSeg){
    for (int i = 0 ; i < iterations ; i++){
        blink(pin1, mSeg);
        blink(pin2, mSeg);
        blink(pin3, mSeg);
        blink(pin4, mSeg);
        blink(pin5, mSeg);
        blink(pin4, mSeg);
        blink(pin3, mSeg);
        blink(pin2, mSeg);
    }
}
```

```
    }  
}  
void setup() {  
    pinMode(pin1, OUTPUT);  
    pinMode(pin2, OUTPUT);  
    pinMode(pin3, OUTPUT);  
    pinMode(pin4, OUTPUT);  
    pinMode(pin5, OUTPUT);  
}  
void loop() {  
    digitalWrite(pin1, HIGH);  
    digitalWrite(pin5, HIGH);  
    delay(550);  
    digitalWrite(pin1, LOW);  
    digitalWrite(pin5, LOW);  
    digitalWrite(pin2, HIGH);  
    digitalWrite(pin4, HIGH);  
    delay(550);  
    digitalWrite(pin2, LOW);  
    digitalWrite(pin4, LOW);  
    blink(pin3, 100);  
    blink(pin3, 100);  
    blink(pin3, 100);  
    blink(pin1, 50);  
    blink(pin2, 50);  
    blink(pin3, 50);  
    blink(pin4, 50);  
    blink(pin5, 50);  
    blink(pin1, 50);  
    blink(pin2, 50);
```



```
blink(pin3, 50);
blink(pin4, 50);
blink(pin5, 50);
blink(pin4, 50);
blink(pin3, 50);
blink(pin2, 50);
blink(pin1, 50);
blink(pin5, 50);
blink(pin4, 50);
blink(pin3, 50);
blink(pin2, 50);
blink(pin1, 50);
digitalWrite(pin5, HIGH);
digitalWrite(pin3, HIGH);
delay(550);
digitalWrite(pin5, LOW);
digitalWrite(pin3, LOW);
digitalWrite(pin4, HIGH);
digitalWrite(pin2, HIGH);
delay(550);
digitalWrite(pin4, LOW);
digitalWrite(pin2, LOW);

digitalWrite(pin3, HIGH);
digitalWrite(pin1, HIGH);
delay(550);
digitalWrite(pin3, LOW);
digitalWrite(pin1, LOW);
digitalWrite(pin4, HIGH);
digitalWrite(pin2, HIGH);
```

```
delay(550);
digitalWrite(pin4, LOW);
digitalWrite(pin2, LOW);
digitalWrite(pin5, HIGH);
digitalWrite(pin3, HIGH);
delay(550);
digitalWrite(pin5, LOW);
digitalWrite(pin3, LOW);
offAll(pines, 5);
delay(250);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
blinkAll(pines, 5, 100);
turnAround(20, 20);
for (int i = 25 ; i < 150 ; i+= 5){
    turnAround(1, i);
}
for (int i = 150 ; i > 15 ; i -= 5){
    turnAround(1, i);
}
turnAround(100, 15);
offAll(pines, 5);
delay(1500);
}
```

Output:

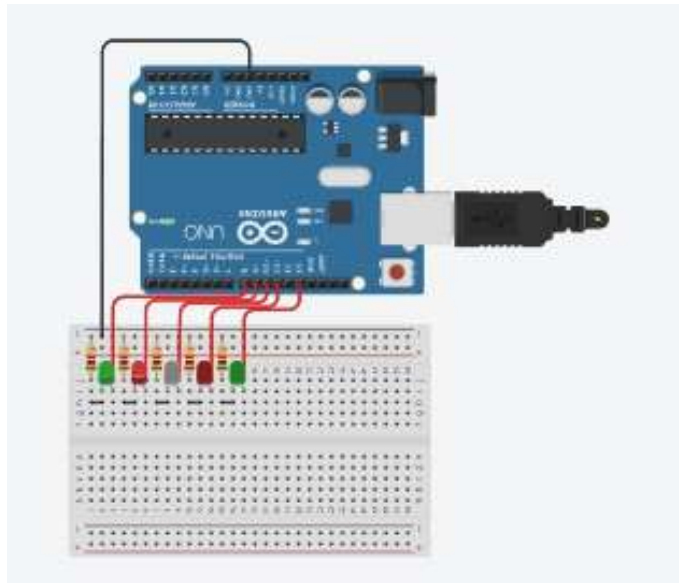


Figure 12.6

D. Controlling LED with a Button

Components Required:

- 1 × Arduino Uno
- 1 × LED
- 1 × 220Ω Resistor
- 1 × Push Button
- 1 × 10kΩ Resistor
- 1 × Breadboard
- Jumper Wires
- 1 × USB Cable

Wiring Instructions:

1. LED Connections:
 - Connect the longer leg (anode) of the LED to Digital Pin 8 on Arduino.
 - Connect the shorter leg (cathode) to GND through a 220Ω resistor.
2. Button Connection:
 - Connect one leg of the button to Digital Pin 2 on Arduino.
 - Connect the same leg to GND through a 10kΩ resistor (pull-down).
 - Connect the other leg of the button to 5V.
3. Powering Arduino:
 - Plug the Arduino Uno into your PC using a USB cable.
4. GND Wire:
 - Connect the Arduino GND pin to the breadboard GND rail.
5. Code to Control LED with Button
6. Testing the Setup:

- Open the Arduino IDE.
- Copy and paste the code below.
- Select Board: Arduino Uno and the correct Port under the Tools menu.
- Click Upload.
- Press the button → the LED should turn ON.
- Release the button → the LED should turn OFF.

Circuit Diagram:

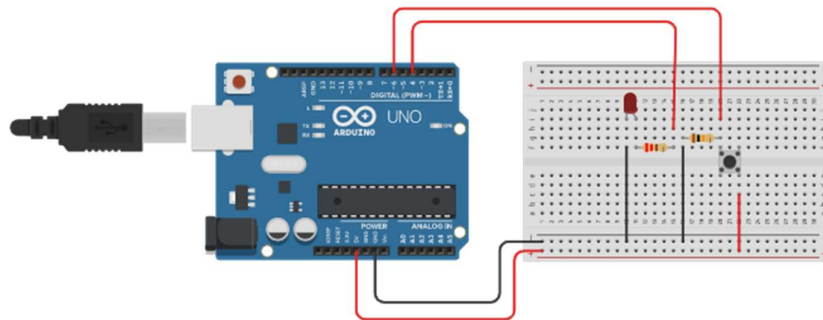


Figure 12.7

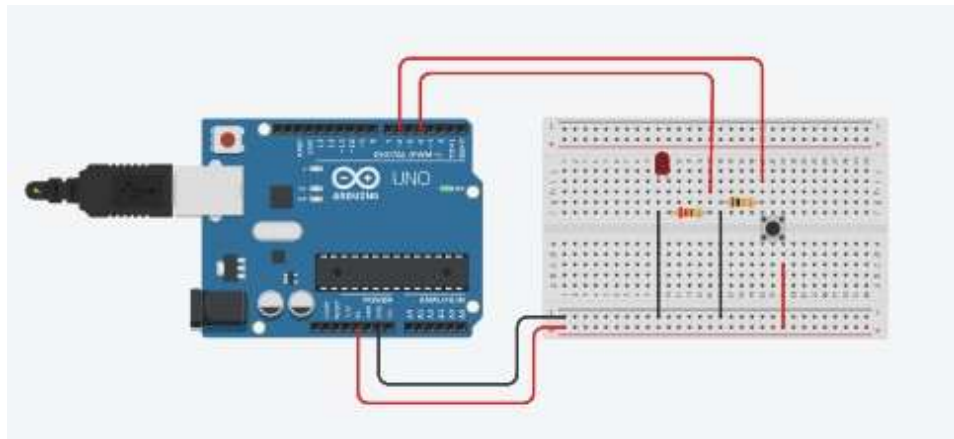
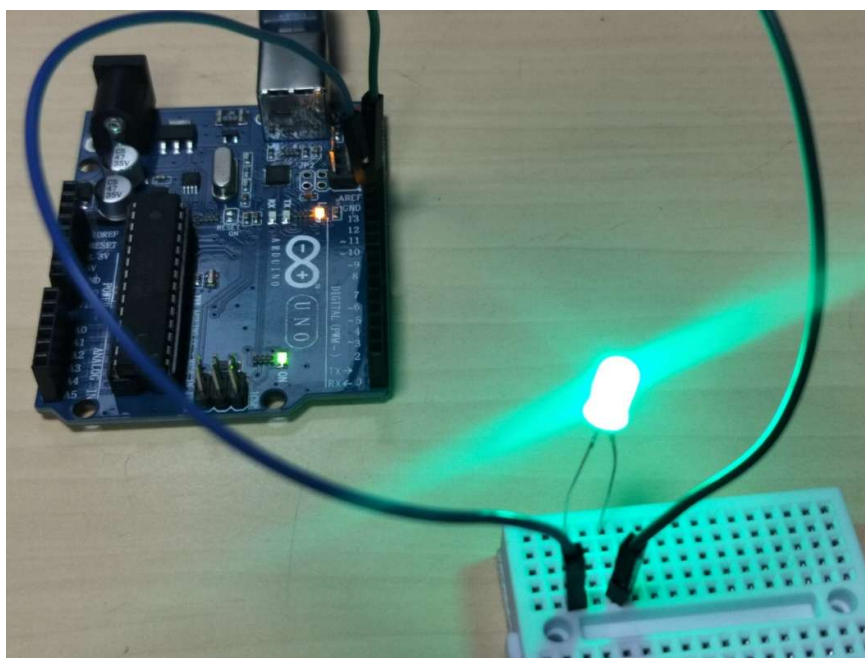
Code:

```
void setup()
{
  pinMode(4, OUTPUT);
  pinMode(6, INPUT);
}

void loop()
{
  if(digitalRead(6)==1){
    digitalWrite(4,HIGH);
  }else{
    digitalWrite(4,LOW);
  }
}
```

}

Output:

*Figure 12.8**Figure 12.9*

Experiment No: 13

AIM: To understand the working of a servo motor or ultrasonic sensor using Arduino UNO.

OBJECTIVE: Learn how to control a servo motor and read data from an ultrasonic sensor using Arduino UNO.

Components Required:

- 1 × Arduino Uno
- 1 × Servo Motor (e.g., SG90)
- Jumper Wires
- 1 × USB Cable

Wiring Instructions:

1. Servo Motor Connections:

- Orange (Signal) Wire → Arduino Digital Pin 9
- Red (VCC/Power) → 5V on Arduino
- Brown/Black (GND) → GND on Arduino

2. Powering Arduino:

- Connect the Arduino Uno to your PC using a USB cable.

Working Principle:

- The servo motor is controlled by sending PWM (Pulse Width Modulation) signals from Arduino.
 - You can set the motor to rotate to a specific angle (0° to 180°).
 - This is useful in robotics, arms, and precision control mechanisms.
3. Testing the Setup:
- Open the Arduino IDE.
 - Paste the above code.
 - Go to Tools → Board → Arduino Uno and select the correct Port.
 - Click Upload.
 - The servo motor will rotate to 0° , then 90° , and then 180° in a loop.

Circuit Diagram:

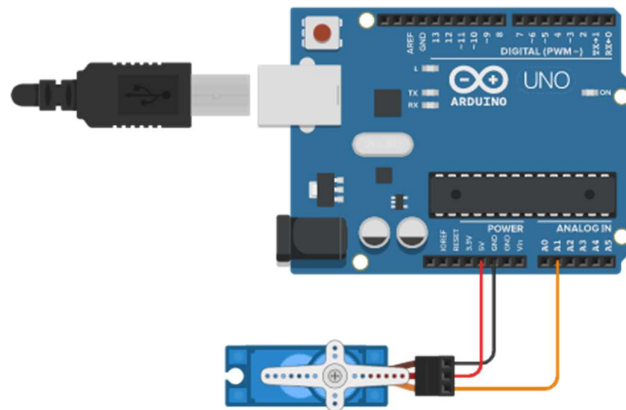


Figure 13.1

Code:

```
#include <Servo.h>

Servo servoBase;

void setup() {
    servoBase.attach(A1);
    servoBase.write(0);
}

void loop() {
    for(int i=0; i<=180; i=i+10)
    {
        servoBase.write(i);
        delay(2000);
    }
}
```

Output:

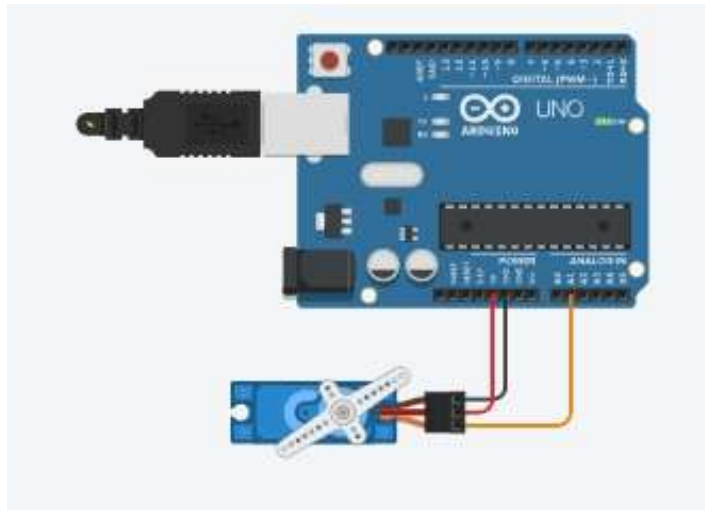


Figure 13.2

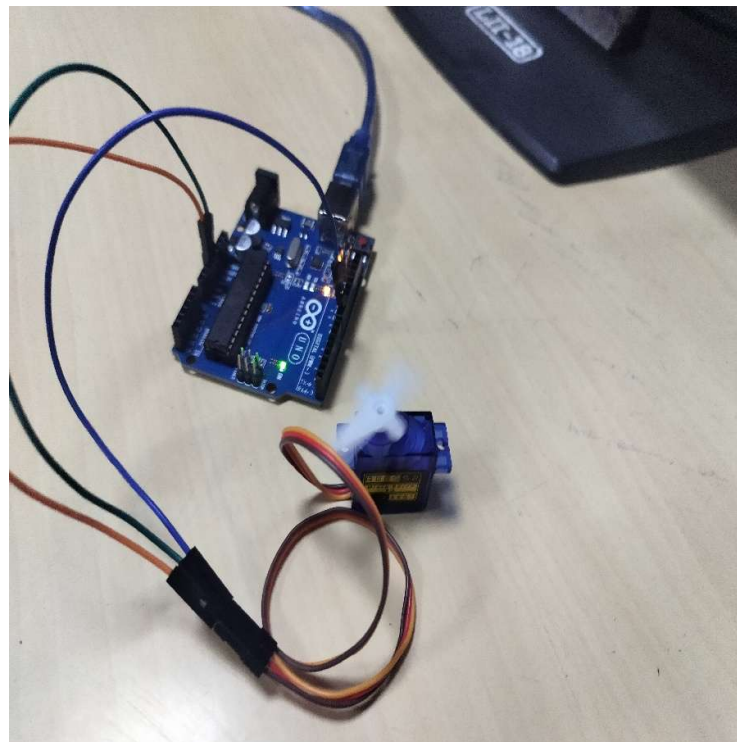


Figure 13.3

Experiment No: 14

AIM: To create a smart dustbin using an ultrasonic sensor and a servo motor with Arduino UNO.

OBJECTIVE: Detect the presence of a user and open/close the dustbin lid using a servo motor automatically.

Components Required:

- 1 × Arduino Uno
- 1 × Ultrasonic Sensor (HC-SR04)
- 1 × Servo Motor (SG90 or similar)
- 1 × Breadboard
- Jumper Wires
- 1 × USB Cable (for Arduino Uno)

Wiring Instructions:

1. Ultrasonic Sensor (HC-SR04):
 - VCC → 5V on Arduino
 - GND → GND on Arduino
 - Trig → Digital Pin 9
 - Echo → Digital Pin 10
2. Servo Motor:
 - VCC (Red wire) → 5V on Arduino
 - GND (Brown/Black wire) → GND on Arduino
 - Signal (Orange/Yellow wire) → Digital Pin 3
3. Powering the Setup:
 - Connect Arduino to PC using USB cable.

Working Principle:

- The ultrasonic sensor detects if someone's hand is close to the dustbin.
- If an object is within a certain distance (e.g., < 15 cm), the servo motor rotates to open the lid.
- After a few seconds, the lid closes automatically.

Testing the Setup:

- Open Arduino IDE.
- Connect Arduino Uno to PC using USB.
- Select Board: Arduino Uno and the correct Port under Tools.
- Paste the code and click Upload.
- Move your hand close to the sensor.
- The lid will open and close automatically.

Circuit Diagram:

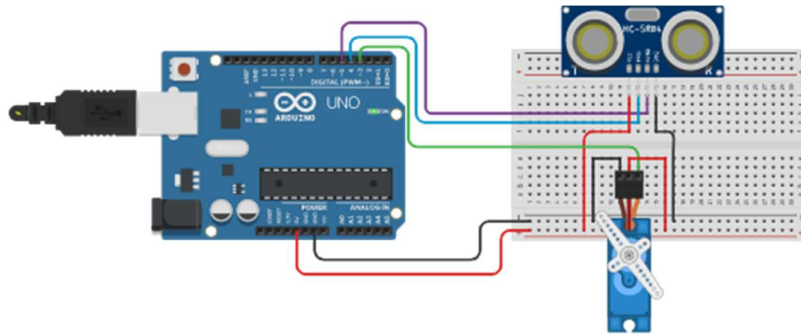


Figure 14.1

Code:

```
#include<Servo.h>

int trigPin = 4;
int echoPin = 5;
long duration_d, dist_d;
long aver_d[3];
Servo Myservo;
int pos;

void setup()
{
  Myservo.attach(3);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  delay(100);
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(5);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(15);
  digitalWrite(trigPin, LOW);
```

```
pinMode(echoPin, INPUT);  
duration_d = pulseIn(echoPin, HIGH);  
dist_d = (duration_d/2) / 29.1;  
if(dist_d<=13)  
{  
  for(pos=0;pos<=180;pos++){  
    Myservo.write(pos);  
    delay(15);  
  }  
  delay(1000);  
  
  for(pos=180;pos>=0;pos--){  
    Myservo.write(pos);  
    delay(15);  
  }  
  delay(1000);  
}  
}
```

Output:

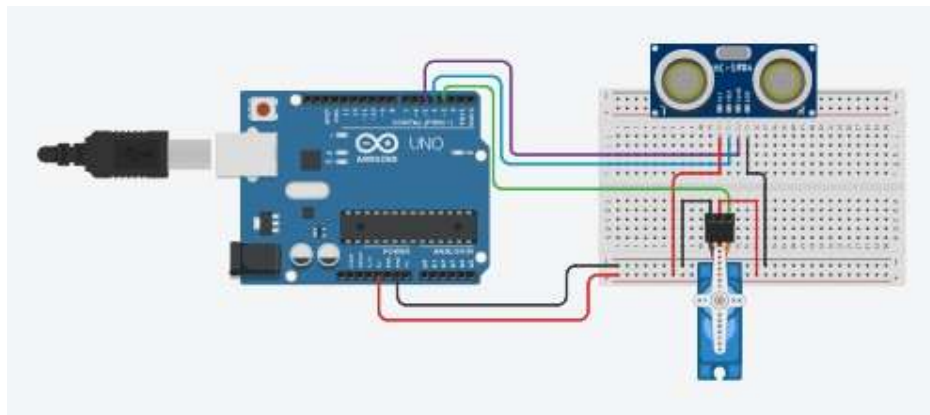


Figure 14.2

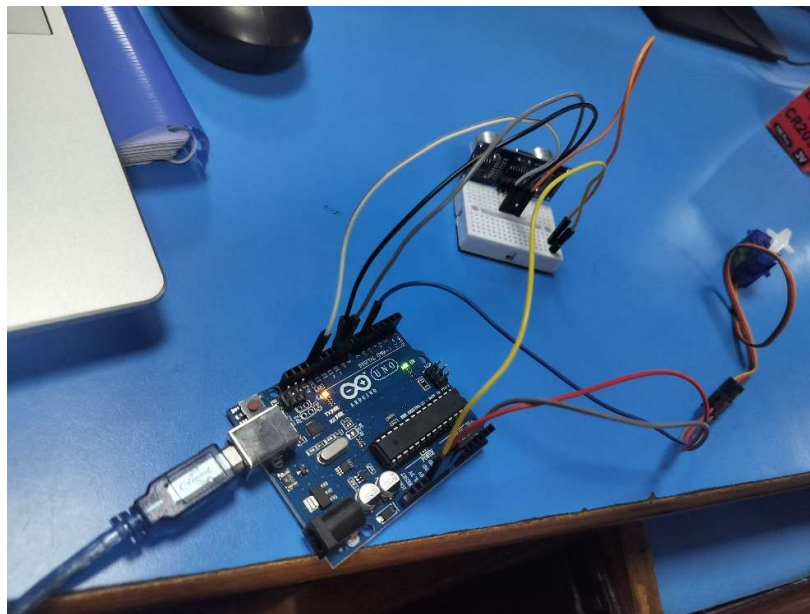


Figure 14.3

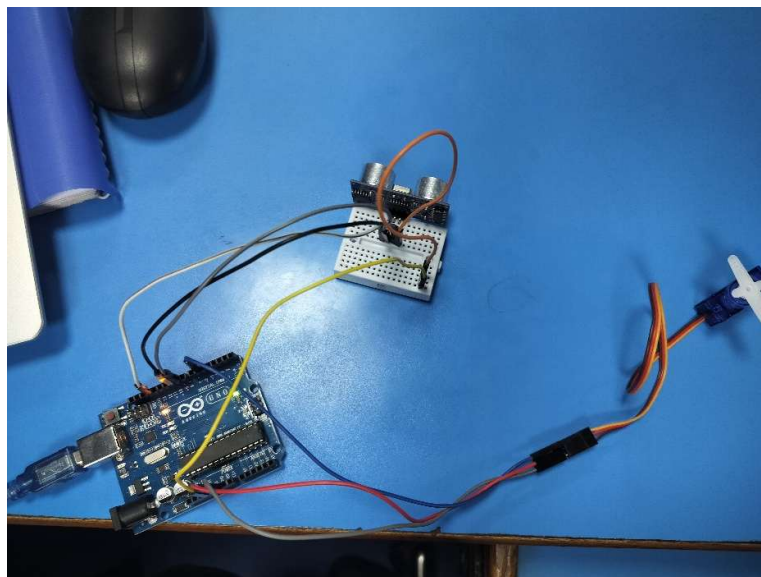


Figure 14.4

Experiment No: 15

AIM: To write the steps to add Blynk libraries for Arduino UNO and create an account on IFTTT for home automation.

OBJECTIVE: Understand how to integrate Arduino UNO with the Blynk cloud and IFTTT platform for IoT-based automation.

1. Install Blynk Library using built-in library manager in Arduino IDE.

To install a new library into your Arduino IDE you can use the Library Manager (available from IDE version 1.6.2). Open the IDE and click to the "Sketch" menu and then Include Library > Manage Libraries.

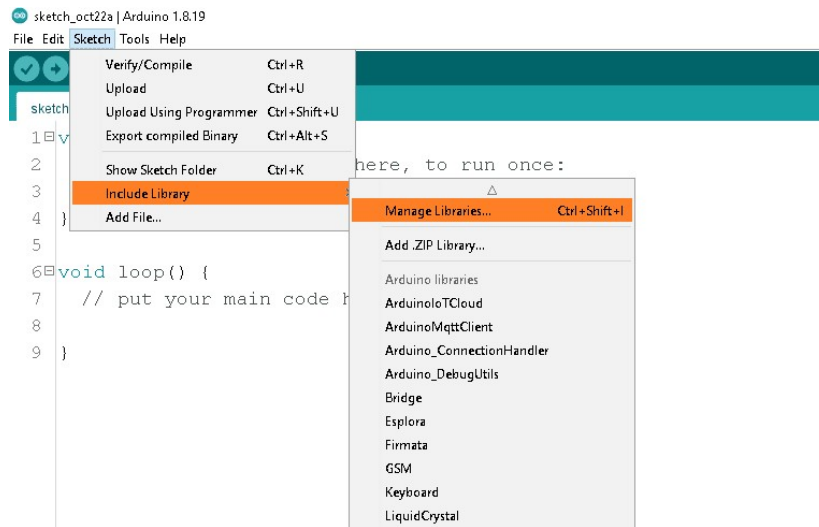


Figure 15.1

Then the Library Manager will open and you will find a list of libraries that are already installed or ready for installation. Search for Blynk library and in the version selection choose the latest version to date.

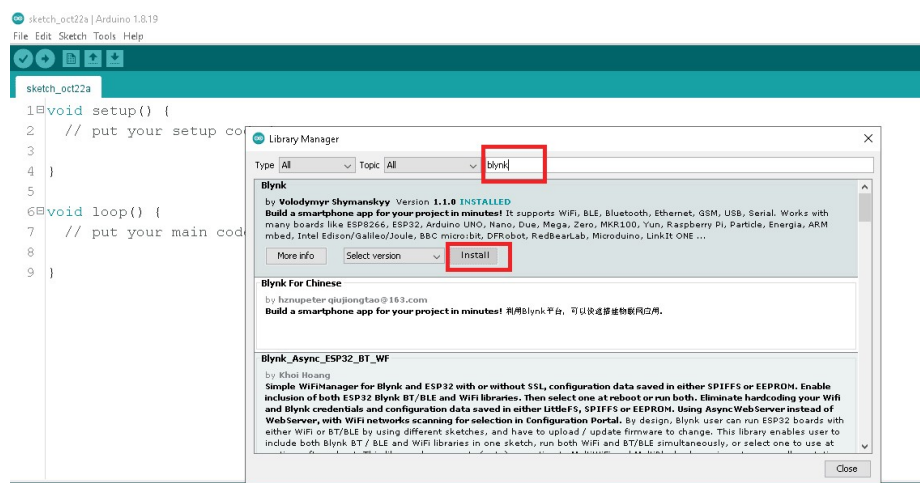


Figure 15.2

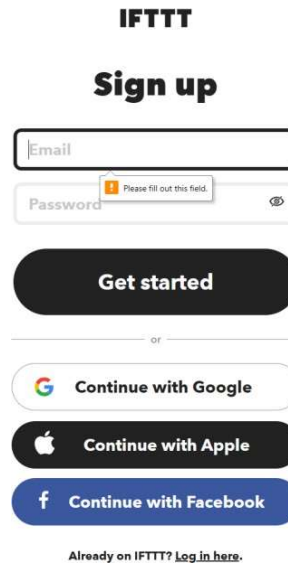
Finally click on Install and wait for the IDE to install the new library. Downloading may take time depending on your connection speed. Once it has finished, an Installed tag should appear next to the Bridge library. You can close the library manager.

You can now find the new library available in the Sketch > Include Library menu.

2. Creating an IFTTT Account and Setting Up for Home Automation

Step 1: Sign Up on IFTTT

- Go to <https://ifttt.com> and click on Sign Up.
- Register using your email, Google, or Apple account



The image shows the IFTTT 'Sign up' page. At the top, it says 'IFTTT' and 'Sign up'. Below this are two input fields: 'Email' and 'Password'. A small error message 'Please fill out this field.' is visible next to the Password field. Below the input fields is a large black button labeled 'Get started'. Underneath this button is a horizontal line with the word 'or' in the center. Below the line are three buttons for social login: 'Continue with Google' (with the Google logo), 'Continue with Apple' (with the Apple logo), and 'Continue with Facebook' (with the Facebook logo). At the bottom, there is a link that says 'Already on IFTTT? Log in here.'

Figure 15.3

Step 2: Create a New Applet

- After logging in, click on Create.



The image shows the IFTTT 'Create your own' applet screen. At the top, there is a 'Cancel' button on the left and a question mark icon on the right. Below this is a banner with the text 'Upgrade for more, faster, better Applets with advanced features. Upgrade'. Underneath the banner, it says 'You're using 0 of 5 Applets'. The main part of the screen shows a blue box with the word 'If' and a plus icon, followed by the text 'Receive a web request'. Below this box is a plus icon, and then a black box with the text 'Then That'. To the right of the 'Then That' box is an 'Add' button with a plus icon.

Figure 15.4

- Click on + This to set up the trigger.
- Search for "Webhooks" and select it.

Choose a service



Figure 15.5

- Choose "Receive a web request".
- Enter an Event Name (e.g., door_opened) and click Create Trigger.

Complete trigger fields



Receive a web request

This trigger fires every time the Maker service receives a web request to notify it of an event. For information on triggering events, go to your Maker service settings and then the listed URL (web) or tap your username (mobile).

Event Name

The name of the event, like "button_pressed" or "front_door_opened"

Create trigger

Figure 15.6

Step 3: Define the Action

- Click on + That to set up the action.



Figure 15.7

- Choose a service (e.g., Email, Notifications, or Smart Home devices).

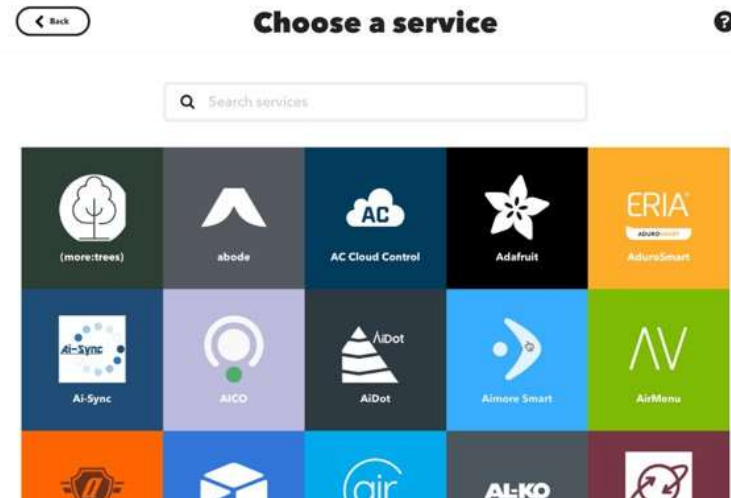


Figure 15.8

- Configure the action as desired and click Create Action.



Figure 15.9

- Finally, click Finish to activate the Applet.

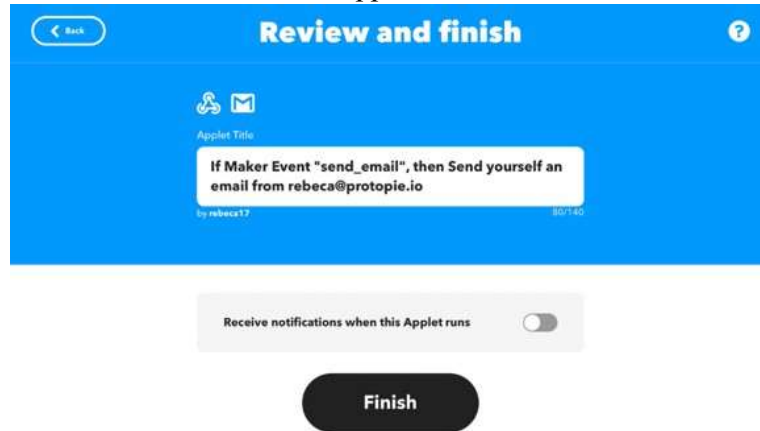


Figure 15.10

Step 4: Obtain Your Webhooks Key

- Navigate to https://ifttt.com/maker_webhooks.
- Click on Documentation in the upper right corner.
- Your Webhook URL will be displayed, typically in the format:
- `https://maker.ifttt.com/trigger/{event_name}/with/key/{your_key}`
- Use this URL in your Arduino code to trigger the IFTTT event.

Experiment No: 16

AIM: To write a program to control a bulb ON/OFF using the Blynk app and Blynk cloud.

OBJECTIVE: Enable wireless control of a bulb using a smartphone by interfacing Arduino UNO with the Blynk IoT platform.

Components Required:

- 1 x Arduino Uno
- 1 x ESP8266 WiFi module (e.g., ESP-01)
- 1 x LED (or relay + bulb)
- 1 x 220Ω resistor (for LED only)
- 1 x Breadboard
- Jumper Wires
- 1 x 3.3V Regulator Module (like AMS1117) — important for ESP8266
- Blynk App on mobile
- Internet connection

Circuit Connections:

- ESP8266 (ESP-01) to Arduino Uno:

ESP8266 Pin	Arduino Pin
VCC	3.3V (from regulator)
GND	GND
TX	Arduino Pin 2 (via voltage divider)
RX	Arduino Pin 3 (through 1k/2k voltage divider)
CH_PD (EN)	3.3V

Note : DO NOT connect ESP8266 RX directly to Arduino TX (5V) — always use a voltage divider (e.g., 2kΩ + 1kΩ) to avoid damaging ESP8266.

- LED (or relay):
 - LED anode → Arduino pin 13 (through 220Ω resistor)
 - LED cathode → GND

Blynk App Setup:

1. Open the Blynk IoT app (not the legacy version).
2. Create a new project:
 - Name: Bulb Control
 - Device: ESP8266
 - Connection: WiFi
3. Copy the Auth Token (sent to your email).
4. Add a Button widget:
 - Connect it to Virtual Pin V0

- Set mode to Switch

Arduino IDE Setup:

1. Install Libraries:
 - Install Blynk library from Library Manager.
 - Install SoftwareSerial (comes pre-installed).
2. Upload The Code
3. Final Testing:
 - Connect everything as shown above.
 - Upload the code to Arduino Uno.
 - Open Blynk app → Tap the button → LED or Bulb turns ON/OFF!

Circuit Diagram:

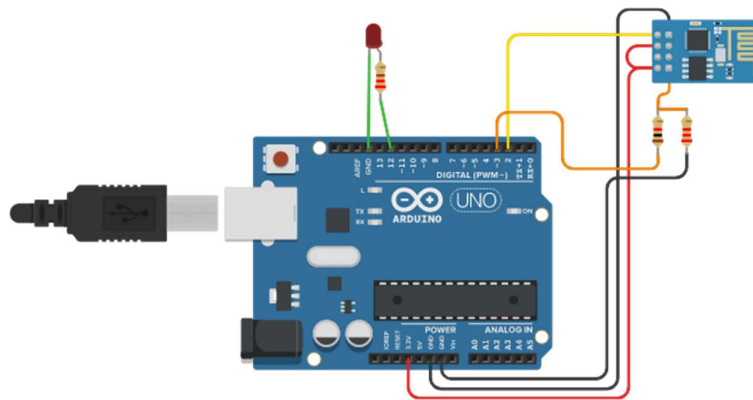


Figure 16.1

Code:

```
#define BLYNK_PRINT Serial
#include <SoftwareSerial.h>
#include <BlynkSimpleEsp8266.h>

char auth[] = "YourAuthToken"; // Blynk Auth Token
char ssid[] = "YourWiFiSSID"; // Your WiFi name
char pass[] = "YourWiFiPassword"; // Your WiFi password

SoftwareSerial EspSerial(2, 3); // RX, TX

void setup() {
  pinMode(13, OUTPUT); // LED pin
  Serial.begin(9600);
```

```
EspSerial.begin(9600);  
Blynk.begin(EspSerial, auth, ssid, pass);  
}  
BLYNK_WRITE(V0) {  
  int pinValue = param.asInt(); // Get value from Blynk App  
  digitalWrite(13, pinValue); // Control LED  
}  
void loop() {  
  Blynk.run();  
}
```

Output :

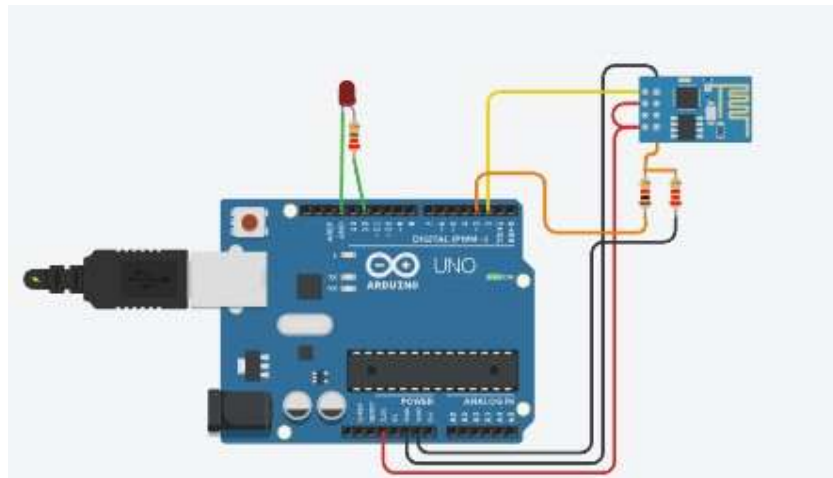
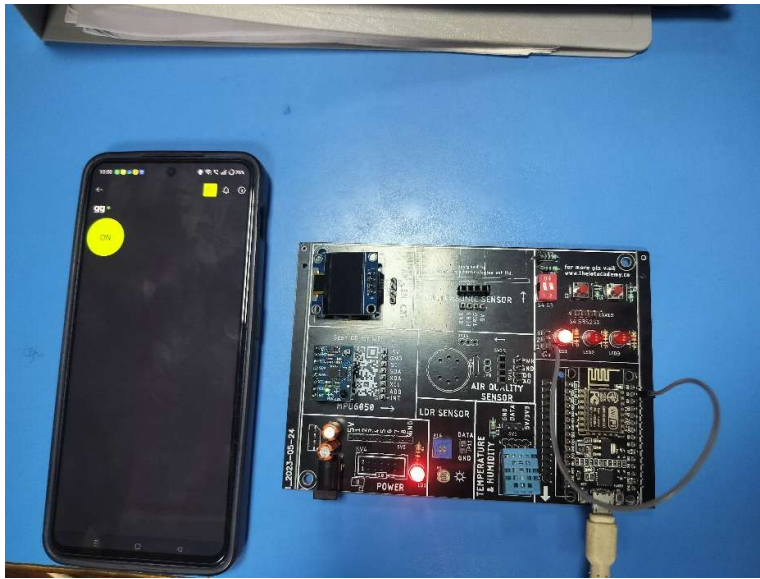
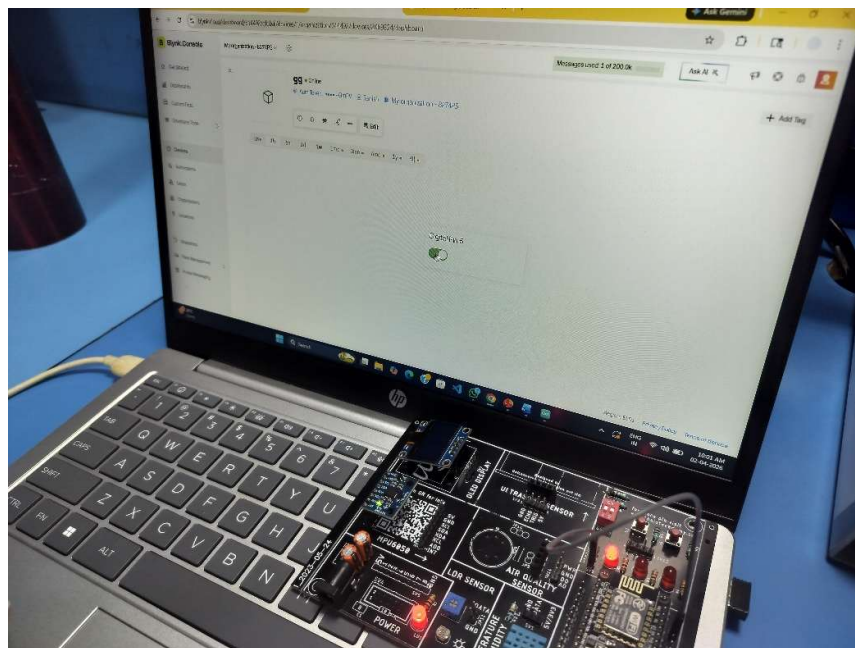


Figure 16.2

*Figure 16.3**Figure 16.4*